SpecExec: Oracle-Grounded Generation and Detection of Speculative-Execution Leaks

Anonymous

***Abstract*—Speculative-execution leaks are found today with hand-written per-ISA gadget patterns, symbolic checkers that each model one speculation mechanism, and slow, CPU-specific hardware fuzzers. We present SpecExec, a pipeline that generates candidate leak gadgets with a class- and ISA-conditioned Transformer, filters them, and adjudicates them with three external oracles: Spectector (symbolic), InvisiSpec on gem5 (execution), and Revizor on a bare-metal Intel i5-8300H. A graph neural network over program dependence graphs classifies assembly into nine classes and supplies the filter's representation. Every leak claim comes from an oracle, never from our own models. On x86 the oracle-in-the-loop generator reaches a confirmed-leak yield of 1.00 for Spectre V1 and 0.98 for Spectre V4, producing about 1,600 distinct leaking gadgets, and a learned pre-oracle gate sustains about one confirmed leak per oracle call. On 423 real hardware violations, a single `lfence` at the speculation boundary removes every stable leak while distant fences do not, giving silicon mitigation labels for L1TF, Spectre V1 and Spectre V4. Much of our contribution is measuring what learned numbers mean: test-set screening, opcode shortcuts, a length confound and a CPU-specific instruction mix each inflated an earlier result. The detector's 95.6% in-distribution accuracy does not transfer to real hardware gadgets, learned instruction features lose to hand features, and no trained model yet beats a non-learned fence rule on the hardware labels. We report these negative results alongside the positive ones.**

I. Introduction

Speculative-execution attacks—Spectre [1], Meltdown [2] and Foreshadow/L1TF [3], MDS [4], [5], Retbleed [6], Inception [7] and BHI [8]—leak secrets through microarchitectural state that the architectural semantics hide. Finding such leaks today relies on hand-written gadget patterns per attack and per instruction set, on symbolic checkers that model one speculation mechanism each, and on hardware fuzzers that are slow and CPU-specific.

The goal of SpecExec is a learned pipeline that, given a description of an instruction set, *generates* instruction sequences likely to cause speculative timing leaks; *filters* them so that only the most promising reach an expensive, precise oracle; and iterates *generate* → *filter* → *oracle* → *retrain* to discover the *smallest* leaking sequences (Fig. 1). A graph-based detector that classifies assembly into eight attack classes and Benign supports the loop.

The difficulty is not building such components but knowing what their numbers mean. Ground truth is scarce: each oracle covers a subset of speculation mechanisms, every oracle we have is x86, and a call costs seconds to a hardware batch slot. Proxies are cheap but easy to over-trust—a classifier's accuracy, or a generator judged by that classifier, can look strong while measuring a shortcut. Two rules therefore govern this paper: every claim that a sequence leaks comes from an oracle external to our models, and every learned number is reported next to the strongest non-learned bar we have.

Findings:

1) **Generation.** On x86 the oracle-in-the-loop generator reaches a confirmed-leak yield of 1.00 (Spectre V1) and 0.98 (Spectre V4), about 1,600 distinct leaking gadgets. A leak-conditioning token trained on hardware labels steers whether a V4 gadget leaks (0.80 vs. 0.04).
2) **Hardware.** On 423 real Revizor violations, an `lfence` at the speculation boundary mitigates every stable case and distant fences do not. L1TF, V1 and V4 violations re-trigger on replay at 100%; the recorded MDS inputs do not (0/6), but fresh contract-fuzzing hits an MDS violation within ≈15 test cases under SMT on *and* off, so the channel is live on this part—the 0/6 is recorded-input fragility, not a closed channel.
3) **Oracle cost.** A learned gate in front of the oracle sustains about one confirmed V4 leak per allowed call (5.5× fewer wasted calls at round 0); a non-learned gate removes 18.2% of unadjudicable candidates, short of the 80% we targeted.
4) **Detector audit.** The detector's 95.6% in-distribution accuracy survives a test-set screening defect, opcode shortcuts and a length confound only partly, does not transfer to real hardware gadgets, and no trained model beats a non-learned fence rule on hardware labels.

Contributions:

1) A declarative ISA specification that drives program-dependence-graph construction, checked against an independent assembler/disassembler oracle. That check, along with follow-up audits, found four bugs that had silently degraded every earlier result: x86 loads classified as stores, size-suffixed mnemonics falling into other, AT&T operand direction, and ARM `blr`.
2) A measured account of what the detector's accuracy means: test-set screening, opcode shortcuts, length confounding, cross-ISA transfer, and synthetic-to-real transfer, each quantified with multi-seed statistics.
3) A hardware-labelling protocol on real Revizor violations. It establishes, on silicon, which `lfence` placements mitigate each class, and it supplies non-learned bars that any mitigation-aware model must beat.
4) An oracle-verified generation loop on x86 that produces distinct leaking gadgets at high yield, with a powered (n=5) diversity/yield analysis of pretraining, and a leak-

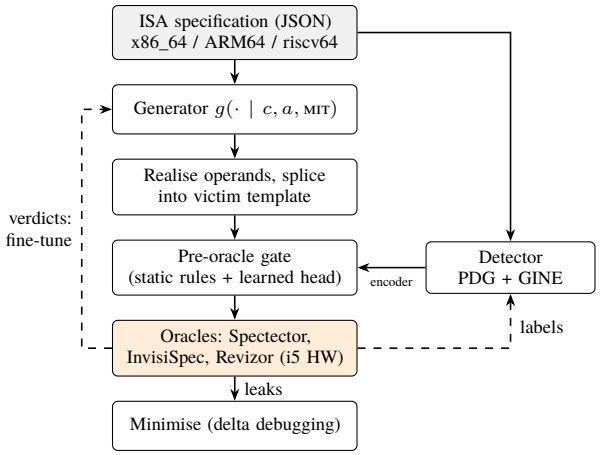


Fig. 1. The SpecExec loop. Solid: data flow; dashed: oracle verdicts retraining the generator and the filter head. Every leak claim comes from the orange box.

conditioning token that gives the first oracle-grounded control over whether the generator emits leaking or mitigated code (Spectre V4: 0.80 vs. 0.04 leak rate per adjudicable gadget; V1's template leaks on both arms by construction, so it cannot show steering).

5) Negative results reported as such: learned node features lose to hand features; the classifier fails as a generation filter; simulator oracles fail for store bypass; oracle-labelled self-training does not manufacture Spectre V2 positives; and a driven speculative emulator does not reproduce hardware labels.

Organisation: Section II covers speculative leaks and the oracles; Section III the design of each component; Section IV datasets and statistical protocol. Section V answers seven research questions (RQ1–RQ7) in pipeline order. Sections VI–VIII discuss lessons, limitations, related work and next steps.

II. Background

A. Speculative-Execution Leaks

A CPU executes past unresolved branches and memory dependences, and squashes the wrong-path work once the prediction resolves. The squash restores architectural state but not microarchitectural state, so a wrong-path load that reads a secret and uses it to index memory leaves a secret-dependent cache footprint that a Flush+Reload probe recovers. The attack classes differ in what supplies the misprediction: the conditional-branch predictor (Spectre V1), the indirect-branch predictor (Spectre V2, and BHI through branch history), return prediction (Retbleed, Inception), store-to-load forwarding (Spectre V4), or a faulting or assisted load (L1TF, MDS). Table I lists the assembly primitive that characterises each class. A *gadget* is a sequence that, under such misprediction, reads a secret and transmits it.

B. Speculation Contracts and Oracles

A speculation contract [9] states what a program's execution is allowed to expose; a program violates it when two inputs the contract deems equivalent produce different hardware observations. The three oracles we use check versions of this property symbolically, in simulation, and on silicon. Every claim that a sequence leaks comes from one of them, never from our own classifier.

TABLE I
Vulnerability classes and characteristic assembly primitives.

Class	Key assembly primitive(s)
SPECTRE_V1	Bounds-bypass: `cmp; j*; ldr [base+idx*s]`
SPECTRE_V2	Indirect branch: `blr xN / jmp *%rN`
SPECTRE_V4	Store-bypass: `str; (spec) ldr` same addr
L1TF	Faulting load + page probe: `shl $12; movq (%r1,%r2)`
MDS	Buffer sampling: assisted/faulting load, `verw`, `movntdqa`
RETBLEED	Return gadget: `push; nop*; pop; ret`
INCEPTION	RSB stuffing: dense `call/ret` pairs (AMD Zen)
BHI	Indirect branch + branch-history conditioning
BENIGN	No speculative transmitter

1) Spectector (symbolic, x86): Spectector [10] checks speculative non-interference over x86 assembly. It is deterministic and runs in a container (Docker for development, Apptainer on the cluster). A per-gadget timeout of at least 300 s is needed, because Spectre V1 data checks take 25–40 s and a 30 s limit times out real leaks. Spectector models conditional-branch (PHT) speculation. In our multi-class runs it can rule on 82% of generated V1 gadgets and 87% of V4 gadgets, but only 15% of V2 and 0.2% of Retbleed gadgets. It cannot model L1TF/MDS faults. The continuous `trace_length` it reports measures the size of the symbolic formula, not the severity of the leak.

2) gem5 and InvisiSpec (execution): Stock gem5 v24 (O3 CPU, classic caches) does *not* leave a misspeculated-load cache footprint: squashed loads are discarded at translation, so Flush+Reload never sees a signal. We rejected it as an oracle. The 2019 InvisiSpec [11] gem5 fork with classic caches does leak Spectre V1: it recovers all 40 secret bytes of a tuned PoC. It does *not* reproduce store bypass. An earlier "V4 leaks 40/40" result was a measurement bug: the harness counted confident cache hits rather than checking that the recovered byte matched the secret. With byte matching, it recovers 0/40 correct bytes. Lesson: a Flush+Reload verdict must compare the recovered value with the secret.

3) Revizor on real hardware (i5-8300H): Revizor [12] runs randomly generated programs on the real CPU through a kernel module and reports a *violation* when hardware traces differ for inputs that a speculation contract says are equivalent. Controls used:

- **MDS and L1TF** violations (3/184 and 3/53 test cases in the first campaign) persist with SMT disabled, ruling out sibling-thread noise.
- **Spectre V4**: a revised configuration (straight-line code, store→load pairs) found 15 violations across 5 seeds. Enabling the speculative-store-bypass mitigation (SSBP MSR)

gives 0; disabling SMT gives 16. Every violating program has a single basic block and no conditional branch, so it is store bypass, not V1.

Revizor runs in batches and only on the bare-metal machine, so it is the most expensive oracle per candidate.

III. SpecExec Design

A. Problem Formulation

Detection: Let $S = (i_1, \ldots, i_n)$ be a window of assembly instructions. The detector is a classifier $f : S \mapsto \mathcal{C}$ with

$$\mathcal{C} = \{\text{BENIGN}, \text{SPECTRE_V1}, \text{V2}, \text{V4}, \text{L1TF}, \text{MDS}, \text{RETBLEED}, \text{INCEPTION}, \text{BHI}\}.$$

The task is assembly-only: no source, IR or runtime information. Exploitability depends on the microarchitecture, so a static label is a *triage* signal for an oracle, not a verdict.

Discovery: A generator $g(\cdot \mid c, a)$ samples sequences for class c and ISA a. An oracle $O(S) \in \{\textsc{leak}, \textsc{safe}, \textsc{unrunnable}\}$ labels a candidate; an oracle call costs from seconds (Spectector) to tens of minutes (InvisiSpec) or a hardware batch slot (Revizor). A filter $r(S)$ orders candidates so that confirmed leaks per oracle call are maximised. The end product is a set of leaking sequences that are *minimal*: removing any instruction stops the leak.

B. ISA Specification and Graph Construction

1) Declarative ISA specification: Classification logic (opcode categories, memory-access types, speculation flags, def/use registers, edge windows, realisation parameters) lives in per-ISA JSON specifications (`spec/x86_64.json`, `arm64.json`, `riscv.json`) over a shared `base` document. A generic engine with no ISA knowledge of its own interprets them. The spec was first exported from the hand-built PDG builder and verified identical on the full corpus (25,942 instruction pairs; 207,113 nodes and 437,231 edges; zero mismatches). Because an exported spec matches its source by construction, that check proves refactoring fidelity, not correctness. Correctness against an independent assembler/disassembler is checked by a repeatable gate, whose findings we report as RQ1 (Section V-A).

2) Program dependence graph: Each window becomes a PDG with one node per instruction. Node features (41 dims): opcode-category one-hot (19), memory-access type (5), register degree (2), and 14 binary speculation flags (`is_branch`, `is_secret_source`, `is_transmitter`, `is_lfence`, …). There are nine edge types:

- DATA_DEP: reaching definitions;
- CONTROL_FLOW;
- SPEC_CONDITIONAL, SPEC_INDIRECT and SPEC_RETURN: from a speculation source to the instructions in its window;
- MEMORY_ORDER;
- CACHE_TEMPORAL;
- FENCE_BOUNDARY;
- RSB_CHAIN: call/return pairs.

Store bypass has no reliable static signature: an SSB edge we built also fired on −O0 Spectre V1, which spills its index through the stack, so we reverted it. A related decision, now settled: under the spec builder, address-based MEMORY_ORDER edges are emitted unconditionally (`memory_order_mode: "address_base"`), which a per-model training flag does not override. The locked model and every baseline were trained with these edges present, so they are kept on; changing the default would invalidate the locked ensemble and all numbers measured against it, for an edge type whose recall effect is small. It is held behind the feature gate rather than left as an open question.

3) Tokenisation and learned instruction features: Instructions are normalised: registers become `<reg>`, immediates `<imm>`, memory operands `<mem>`/`<mem-idx>`, and neutralised call targets `<fn>`. Three vocabularies exist:

- *mnemonic*: literal mnemonics; 92.7% out-of-vocabulary on RISC-V;
- *canonical*: 53 ISA-neutral operation names declared in the spec (`addq/adds/addi` → `ADD`); 7.9–9.1% out-of-vocabulary on RISC-V;
- *neutral*: 0.0% out-of-vocabulary on RISC-V.

A masked-language-model Transformer [13] trained on x86 and ARM yields per-instruction embeddings (`spec/mlm_neutral.pt`). Token *frequencies* still identify the ISA (0.98), so the locked model adds an adversarial architecture head (Section III-C).

4) Inline features: The 58 handcrafted inline features (opcode fractions, structural ratios, pattern densities, primitive flags, page-probe detection; Appendix D) include many ISA literals. On RISC-V, 43 of the 58 are identically zero. A *neutral* subset of 9 features that fire on every ISA is used in the cross-ISA configuration.

C. Detector

1) Architecture: The backbone is GINE [14]:

$$h_v^{(k)} = \text{MLP}^{(k)}\Big((1+\varepsilon^{(k)})h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \text{ReLU}\big(h_u^{(k-1)} + s_t e_{uv}\big)\Big)$$

with learnable per-edge-type scales s_t. It uses three layers, hidden size 128, batch normalisation, residual connections, a gated virtual node, Jumping-Knowledge concatenation [15] and attention readout. The graph embedding is fused with an MLP over the inline features, global features (5) and an architecture embedding (8), then classified by a two-layer MLP. The loss is class-weighted cross-entropy plus supervised contrastive [16] (13 hard-negative pairs, warm-up) plus an auxiliary inline-feature head. The optimiser is AdamW [17] (10^{-3}), with early stopping on a group-stratified validation split (the group is the source file).

Configurations:

- **B1** (in-distribution reference): hand node features, spec builder, architecture embedding, 58 inline features.

- **No-hand baseline**: architecture embedding, no inline features, trained on the trigger-masked set (Section V-B2).
- **Locked cross-ISA model** (`models/locked_classifier.json`, locked 2026-09-25): learned neutral-MLM node features, the 9 neutral inline features, NOPs dropped, node dropout 0.1, an adversarial architecture head ($\lambda = 1$), and length-matched training data. It is a 5-seed ensemble (mean softmax), loaded with SHA-256 verification.

2) Augmentation: Nine augmentation transforms preserve the attack invariant, and a 143-test suite verifies them: register rename, NOP insertion, local swap, immediate perturbation, stride-synonym swap, idiom substitution, branch-polarity flip, barrier upgrade (never downgrade) and slice recomposition. The suite found six implementation bugs (Appendix C). Separately, *trigger-masked* augmentation replaces class-defining opcodes so that the model cannot rely on them (Section V-B2).

D. Generator

1) Model and realisation: A decoder-only Transformer [18] ($d = 128$, 3 layers, 4 heads) is trained by next-token prediction on normalised instruction tokens. Each sequence is prefixed with a class token and an architecture token, so sampling from $[\langle \textsc{cls} \rangle_c, \langle \textsc{arch} \rangle_a]$ conditions on both. Base training uses 5,532 real gadget records. Optional self-supervised pretraining uses 200k compiled C function bodies (AnghaBench [19]) for x86_64, ARM64 and riscv64. A spec-driven *realiser* instantiates operand placeholders with registers, immediates (biased toward attack constants such as 64 and 4096) and addressing modes from the target ISA's spec. For oracle checks, the realised body is *spliced* into a fixed, class-specific misdirection template (e.g. a bounds-check bypass) that ends in a cache-line-scaled probe write.

Conditioning: A reference classifier fit on real data finds the requested class 7.4× more often than the class prior (class only), and 5.7× with class and architecture. These numbers come from a classifier, not an oracle. An ISA-purity figure of 96–98% is likely optimistic: the purity metric was blind to ARM load/store mnemonics (`ldr`, `str`, …) appearing in x86 output.

Leak-conditioning: The class token conditions on *which* vulnerability class to emit, not on whether a gadget actually leaks: the attack-class training records are labelled by provenance, not by an oracle verdict, so a class-conditioned sample is only class-*shaped*. To steer the generator toward *unmitigated* code we add an optional third conditioning token $\langle \textsc{mit} \rangle \in \{\textsc{leak}, \textsc{safe}, \textsc{unknown}\}$ (prefix order class, arch, mit), trained on the hardware mitigation labels of Section V-C2: a leaking variant is labelled LEAK, its mitigated twin SAFE *with the class token preserved* so the pair differs only in mit, and the 725**[TODO: reconcile with the 423 violations of Section IV-A4]** hardware-confirmed pool gadgets LEAK; every unverified legacy record is UNKNOWN, so the token never asserts a leak status we did not confirm. Base gadgets are held out by violation id across *every* source—the leaking variants are transforms of the pool originals, so a held-out base is withheld from the pool too. The token is additive: with it disabled the generator is byte-identical to before.

Oracle-in-the-loop training: Rejection-sampled fine-tuning repeats four steps: sample, realise and splice, check with the oracle, then fine-tune on the confirmed leaks (learning rate 10^{-4}). Every realised sample, including unrunnable ones, is logged, so yield has a denominator and diversity can be audited.

E. Oracle Harness and Coverage

Verification coverage, not data, is what bounds this work per class, so it is worth stating exactly where the boundary falls and why.

Revizor cannot be configured for indirect-branch or return speculation: Its contract-execution clauses are conditional-branch misprediction, store bypass, and fault or assist transients (`seq`, `cond`, `bpas`, `meltdown`, `nullinj-*`, `seq-assist`, `vspec-*`). There is no indirect-branch clause and no return-prediction clause, and our four demo configurations map one-to-one onto the four usable clauses—so the four missing classes are not an oversight in our configurations but outside the tool's contract model. The limitation is deeper than a missing clause: the x86 configuration also comments out `BASE-CALL`, `BASE-RET` and `BASE-UNCOND_BR` as "Not supported: Complex control flow", so calls and returns are excluded from the generated instruction pool as well. No CPU purchase changes this.

Upstream Spectector models conditional branches only: Our oracle invokes it as `run-spectector -a noninter`, which is why it rules on about 82% of SPECTRE_V1 candidates and 87% of SPECTRE_V4 but only about 15% of SPECTRE_V2 and 0.2% of RETBLEED.

A published extension covers part of the gap: Spectector-Combined [20] extends the analysis with further mechanisms selected per run by a `--version` flag: conditional branch, indirect branch, store forwarding, return speculation, and straight-line speculation, combinable except where two mechanisms speculate on the same instructions. In principle this makes SPECTRE_V2 symbolically adjudicable with no hardware at all, which would be the single largest coverage gain available to this project. Our side is integrated and tested—the flag (`-v`, which is undocumented in the tool's usage text and discoverable only from the fork's own test scripts), a per-class mechanism map that deliberately omits MDS, L1TF and BHI because it models none of their mechanisms, and a separate container image so existing V1/V4 results stay reproducible against the oracle that produced them.

The fork's artifact does not run against current Ciao; pinning Ciao to the fork's era fixes it, and our call path adds the flags the fork's own scripts use for the extended analyses, gated so the upstream V1/V4 path is byte-identical to before (Appendix B).

TABLE II
LEAK-VERIFICATION COVERAGE BY CLASS. "DETECTION ONLY" MEANS THE CLASSIFIER ASSIGNS THE CLASS BUT NO ORACLE CONFIRMS A LEAK—A TOOLING LIMIT, NOT A PROPERTY OF THE DATA.

Class	Verification available (x86)
SPECTRE_V1	Spectector + Revizor `cond` (replay 3/3)
SPECTRE_V4	Spectector + Revizor `bpas` (SSBD control; replay 14/14)
MDS	Revizor `seq-assist`: recorded violations do not replay (0/6), but fresh fuzzing re-finds one in ≈15 cases (SMT on/off): channel *live*
L1TF	Revizor delayed-exception (replay 6/6)
SPECTRE_V2	Spectector-Combined, **76.6% adjudicable** (Tab. IX)
RETBLEED	Spectector-Combined return model, pending the return-model question
INCEPTION	none: AMD Zen hardware
BHI	none: needs enhanced-IBRS silicon

That the only published extension closing our largest class gap is a 2022 prototype needing an era-matched toolchain to run at all is itself a finding about why the gap persists.

One question must be settled before Spectector-Combined is used for RETBLEED: whether its return model covers the return-buffer-underflow fallback to the branch target buffer that defines that attack on Intel, or only plain return speculation. Until that is answered, a RETBLEED verdict from it would be unsupported.

What remains genuinely unreachable: INCEPTION is an AMD Zen mechanism and BHI needs a part with enhanced IBRS; on the Coffee Lake machine used here, branch-history injection falls under ordinary branch-target injection and is not separately testable. Those two are hardware-blocked and are scoped out of this paper's verification claims rather than left looking unfinished. Table II summarises the resulting position.

1) Emulator-driven speculation, where no oracle exists: Every oracle above is x86. The generator emits ARM64, and the ISA specification, tokeniser and realiser also cover riscv64, so "the generator cannot write a leaking ARM64 gadget" and "we have no ARM64 oracle" were indistinguishable. A CPU emulator does not by itself fix this: Unicorn, like the QEMU it derives from, reproduces only the architectural effect of each instruction and has no branch predictor and no caches, so it can never be *asked* whether a sequence leaks.

We therefore drive the speculation rather than observe it (`oracle/spec_emulator.py`). For each conditional branch, the emulator runs to the branch, snapshots the CPU, executes the branch alone to learn the resolved successor, restores the snapshot, sets the program counter to the *other* successor, and runs a bounded window of instructions while recording the cache lines touched by loads and stores and the instruction addresses fetched. The verdict follows the same property Spectector checks symbolically and Revizor checks against hardware, evaluated concretely:

> Given two inputs that differ *only* in a designated secret region, if their non-speculative observations are identical but their speculative observations differ, the sequence leaks the secret through speculative execution.

Three details carry the result.

- **Inputs differ only in the secret.** Two independently random memory images essentially never produce identical architectural observations; an earlier version compared full random inputs and returned "no architecturally equivalent pair" for 20 of 25 real violations, measuring nothing. The public region is therefore byte-identical across the pair and only a secret region varies. That region sits above the 8 KiB window Revizor's generated programs mask their addresses into (`and reg, 0b1111111111111`), so an architectural access cannot reach it while a misspeculated, unmasked one can—the standard Spectre-V1 arrangement.
- **Architectural equality is required**, which is what separates a speculative leak from an ordinary architectural data flow. When it fails, the secret is observable with no speculation at all; that is reported as a distinct ARCH_LEAK outcome and never counted as a speculative leak.
- **Barriers are modelled explicitly.** Unicorn executes `lfence` as a no-op, because it has no speculation to serialise, so a forced window runs straight through a fence and reports a mitigated gadget as leaking—we observed exactly that. A speculative window now stops at the first serialising instruction, which is the defining semantics of a fence under a speculation contract and matches Spectector's translation of `lfence` to its `spbarr` barrier. Per ISA: x86 `lfence/mfence/sfence/cpuid`, ARM64 `dsb/isb/csdb`, riscv64 `fence`.

Scope, which is narrow: Only conditional-branch (PHT) misprediction is modelled. Store-to-load forwarding (V4), return prediction (Retbleed, Inception), indirect prediction (V2, BHI) and faulting or assisted loads (L1TF, MDS) are not, so a SAFE verdict means "no conditional-branch speculative leak", never "no leak". Only the first execution of each branch is explored, and inputs are concrete, so the check under-approximates: a LEAK verdict is worth considerably more than a SAFE one.

Known-answer behaviour: On hand-written gadgets the emulator gives the textbook answers on *both* x86_64 and ARM64: a Spectre-V1 shape leaks; the same shape with a barrier between the guard branch and the secret read is safe; and a barrier placed after the transmitter still leaks. That last discrimination is the one a position-based rule cannot make, and it is the one the hardware labels of Section V-C2 establish on silicon. ARM64 uses `dsb sy; isb`, the same barrier Revizor's own fence pass emits for that ISA.

F. Pre-Oracle Gate and Learned Filter

1) A non-learned gate in front of the oracle: Before any learned filter, most of what the generator emits is not a safe-or-leaking gadget at all: it is a gadget the oracle *cannot run*. Spectector returned UNRUNNABLE for 456 of 1,987 generated Spectre V1 samples and 511 of 600 Spectre V2 samples, each costing a full oracle call (up to 300 s of symbolic execution) to learn nothing about speculation. We therefore gate candidates

with three fixed, non-learned stages (`gen/precheck.py`), cheapest first.

A. Assembles. The whole sequence is assembled as one unit by `llvm-mc`, through the same independent assembler oracle used elsewhere in this work. Available on all three ISAs.

B. Oracle front end. x86 only: can Spectector's x86 parser accept every instruction it *reaches*? This reimplements Spectector's own acceptance rule from its own source. Its table of 124 `ins/4` facts is extracted from the pinned container with provenance recorded (`oracle/extract_spectector_table.py`), and a mnemonic resolves as a stem plus up to two size-suffix characters from {b,d,l,q,s,t,w} whose arity must match the operand count. Nothing here is fitted to the verdicts the stage is later measured against.

C. Emulates. The sequence executes in Unicorn without an unhandled fault.

Reachability is what makes stage B work: Spectector reports unsupported instructions *per analysed path*, and our oracle wrapper treats a nonzero count as unrunnable. Generated sequences are full of mid-sequence `ret`s, so an instruction Spectector cannot parse is usually in dead code and never counted. Over the 4,379 committed loop samples, an unparseable instruction appears *anywhere* in 176 of 2,355 leaks (7.5%) but on the straight-line reachable prefix in only 15 (0.6%), while still appearing on that prefix in 303 of 1,746 unrunnables (17.4%). Inspecting the whole sequence would have discarded two thirds of the leaks; inspecting the reachable prefix is what makes the stage usable. Stage C emulates the same prefix, for the same reason: a mid-sequence `ret` pops an uninitialised stack and jumps to a garbage address, which is not a defect of the generated code.

Two concessions in stage C, both recorded: Pages are mapped on demand, because the generator routinely clobbers a register with data and then dereferences it; a fixed map would fail almost everything on an address a real harness would have made valid, and Revizor solves the same problem by masking addresses into its sandbox. The count of demand-mapped pages is reported per candidate so the concession is visible. Consequently OK means "decodes and executes as machine code", not "would run usefully in a harness".

2) Learned leak filter: The `rank/` package reuses the locked classifier as a frozen encoder, with a forward hook on the fused representation. A regression head with MC-dropout uncertainty sits on top, candidates are chosen by UCB, and the data is split by token-content hash. Because the classifier's own output is not a usable filter (Section V-E), the target is $P(\text{runnable}) \cdot P(\text{leak} \mid \text{runnable})$. It sits behind a cheap non-learned pre-check (assembles, uses only instructions the oracle supports), and it is evaluated prospectively: train on rounds $\leq r$, test on round $r{+}1$ of held-out seeds. The metric is confirmed leaks per oracle call, against random ordering and against the within-round length bar (0.59). The head-room is largest early: a filter's best gain over random ordering is about $1/\text{leak rate}$, about $5\times$ for V4 round 0 and about $1.14\times$ for V1 by round 3.

G. Minimisation

The goal is the smallest leaking sequences. For real violations we run Revizor's instruction-removal pass, which deletes instructions one at a time from the end and keeps a deletion if the violation survives. This gives, for each violation, the instructions it needs. For generated leaks, the plan is delta debugging [21] with the oracle as the test, with removals ordered by a learned probability that the leak survives, in the spirit of ProbDD [22], to save oracle calls.

IV. Evaluation Setup

A. Datasets

1) Synthetic and compiled corpus (in-distribution):

Internal handcrafted vulnerabilities: Hand-authored C files encoding canonical patterns per class (`c_vulns/`), compiled with GCC and Clang at O0–O3 for x86_64 and ARM64. Only about 21 of them are complete, runnable proofs of concept; the remaining ∼1,400 `.s` files and 530 variant `.c` files are pattern fragments without a secret-to-transmitter data flow. This is why the oracles in Section II-B matter: most training gadgets were never executable leaks.

Compiler-synthesised samples (`p9_*`): Known-vulnerable kernel and library code compiled with GCC and Clang for x86_64 and AArch64 at O0 and O1.

External sources: Google SafeSide [23] (200 samples, compiled for `x86_64/arm64`, O0–O3); the Spectector benchmarks [10] (101 vanilla Kocher variants [1]); and up to 300 FastSpec [24] synthetic V1 gadgets, used for training only.

Benign code: Benign assembly from 1,031 GitHub repositories, compiled for ARM64. The original corpus contained *no x86 benign code*; Section V-B4 shows the consequence (x86 benign false-positive rate 98.4%) and the fix: real x86 and ARM benign functions compiled from mbedTLS, with a disjoint slice held out for validation.

Splits: The locked test set (1,670 records; frozen since pipeline v53) and the v54/v55 training sets are summarised in Appendix C. A length-matched, multi-scale training set (`v54_train_lenmatch_v4s`, 18,897 records before V4-family additions) fixes the length confound of Section V-B4.

2) Test-set screening defect and the unscreened split: The v53 dataset builder applied a *label-conditioned* filter (`has_train_attack_signal`) to the test pool, three lines after a comment saying it did not. Every locked test record therefore satisfies a hand-written rule keyed on its own label (e.g. an MDS record must contain `verw`, `movntdqa` or `clflush`): 100% conformance for every class, against 41% (RETBLEED) and 69% (INCEPTION) on an unscreened control. This is selective data snooping in the sense of Arp et al. [25], and it guarantees a shortcut on the test set.

We fixed the builder: the label-conditioned filter now raises if it is called on a non-training split. We also rebuilt an

TABLE III
ACCURACY ON THE LOCKED (SCREENED) TEST SET VS. THE REBUILT UNSCREENED SPLIT.

Model	Locked	Unscreened	Δ
Hand-58 (RF)	95.15%	90.19%	−4.96 pp
Spec-42 (RF)	96.90%	93.18%	−3.72 pp
GINE (spec builder)	92.75%	92.43%	−0.32 pp

unscreened test split (1,017 records) from the pre-filter pool, using only label-independent quality criteria. Table III gives the inflation. The flat feature models lose 4–5 pp; the GINE graph model loses 0.3 pp, because the graph structure survives the screening.

3) Cross-ISA data:

RISC-V corpus: 498 files cross-compiled for riscv64. An ISA-independence test on canonical-op bigrams (Section V-B4) shows this corpus is *transliterated from ARM64*, not written for RISC-V. In all 6 shared classes it is closer to ARM than ARM is to x86 (sign test $p = 0.016$). We therefore treat it as a stand-in, not as idiomatic RISC-V. A held-out version with corrected labels (`riscv_loio_corpus_v2`, 248 records, 27 attacks) is used for evaluation.

Real RISC-V proofs of concept: 11 records from 4 published RISC-V PoCs (BOOM RTL attacks; T-Head C910 silicon), stamped validation-only. This is the only RISC-V evidence independent of our own corpus. In-order RISC-V cores (SiFive U74, T-Head C906/C908) are empirically not vulnerable, so claims are scoped to out-of-order cores.

4) Real hardware violations (Revizor, i5-8300H): Revizor [12] fuzzing campaigns on a bare-metal Intel i5-8300H (Coffee Lake) produced **423 unique violating programs**: Spectre V1 130, L1TF 155, MDS 83, Spectre V4 55. Each class comes from its own Revizor configuration (instruction categories and contract). A seed ledger prevents reusing a program-generator seed, because a reused seed regenerates byte-identical programs. Violating programs (Intel syntax, `r14`-relative sandbox) are converted to the pipeline's AT&T format with an assemble/disassemble round trip (`clang + llvm-objdump`). The held-out split is by violation (group), with disjoint generator seeds: V1 54, L1TF 64, MDS 33, V4 22. A leakage audit (exact-sequence, group, seed and near-duplicate checks; maximum held-out-to-train similarity 0.76) passes.

B. Statistical Protocol

Detector numbers are means over at least five seeds; single-seed results are not reported as findings. Comparisons between configurations use ten paired seeds with paired tests, and equivalence claims a pre-registered TOST test [26]. Proportions carry Wilson 95% intervals. Held-out RISC-V and hardware sets use cluster-bootstrap intervals over source families, because records from one family are not independent. For RISC-V the grouping is severe: 496 records collapse to 22 source families (Kish effective $n = 7.34$, interval inflation $3.5\times$), and three classes have a single family, so their intervals are undefined rather than narrow. **[TODO: propagate the group-aware intervals into the RISC-V numbers of Section V-B4.]** Finally, every learned number is reported next to the strongest non-learned bar available—sequence length, a bag-of-opcodes logistic regression, or a fence-position rule.

C. Compute environment

The learned components train and evaluate on a university SLURM cluster (GPU nodes); all cluster jobs run through `sbatch`, never on a login node. The symbolic oracles run in containers—Spectector under Docker for development and Apptainer on the cluster, and the Spectector-Combined fork with its Ciao toolchain pinned to the fork's era (`v1.22.0-m1`); the combined image self-tests its V2 example on build and is staged to node-local scratch before use. InvisiSpec runs as a 2019 gem5 fork with classic caches. All hardware measurements use one bare-metal Intel Core i5-8300H (Coffee Lake) with Revizor and its kernel module, under both SMT settings. The cross-ISA speculative emulator is our own Unicorn-driven harness. Reported detector numbers are multi-seed (5 seeds) unless stated otherwise.

V. RESULTS

We answer seven research questions in pipeline order: whether the ISA specification is correct (RQ1), what the detector's accuracy measures (RQ2), what hardware says about mitigations (RQ3), whether the generator produces oracle-verified leaks (RQ4), whether oracle calls can be saved (RQ5), whether discovered leaks are minimal (RQ6), and whether a driven emulator can stand in for an oracle off x86 (RQ7).

A. RQ1: Is the ISA Specification Correct?

For correctness, each instruction is assembled with `llvm-mc`, and its control-flow class is read back with `capstone`. Agreement was 98.80% at first (274 defects), and is 99.86% on the current specs. A repeatable gate (`scripts/run_feature_gate.sh`) must pass before any spec or feature change is trusted. The gate and the audits that followed it found:

- **ARM** `blr` indirect calls fell into OTHER. Fixing it raised Spectre V2 recall from 76% to 94% (+1.32 pp accuracy).
- **x86 loads classified as stores.** In AT&T syntax one mnemonic (`movq`) serves both directions, and the store rule was checked first. 58.5% of unambiguous x86 memory instructions were misclassified, which made the secret-source and transmitter flags effectively dead on x86 (0.01% vs. 1.00% on ARM). Fixed by an operand-order field in the spec (0.87% after the fix). The control-flow oracle gate *cannot see this class of bug*, because it covers only CALL/RET/JUMP.
- **Size-suffixed mnemonics** (`addq`, `pushq`, …) missed word-boundary patterns: 28% of all instructions landed in OTHER. Fixed to below 0.3% on all three ISAs.
- **AT&T destination direction** (the first register taken as the destination), sub-register aliasing (`eax/rax`, `w0/x0`,

a0/x10), and data dependence from the last three definitions instead of the reaching definition. A probe classifier that predicts the ISA from each input channel dropped from 0.98 to 0.82 (categories) and from 0.86 to 0.75 (graph) after these fixes. The remaining ISA signal is real (e.g. RISC-V's fused compare-and-branch).

Every result produced before these fixes is re-measured or withdrawn. The in-distribution accuracy in Section V-B1 is on current code.

B. RQ2: What Does Detector Accuracy Measure?

1) In-distribution: On the locked test set, retrained on current code (after all Section V-A fixes), B1 reaches **95.62 ± 0.67%** accuracy over 5 seeds. An earlier multi-seed figure, 96.14 ± 1.59% (macro-F1 85.6), predates the x86 load-/store fix; the single best run (97.07%) is the top of that range, not a typical result. The locked cross-ISA ensemble trades in-distribution accuracy for transfer: 92.45%. Two caveats bound these numbers:

- **Screening** inflates the flat models by 4–5 pp and GINE by 0.3 pp (Table III).
- **Leakage-controlled splits.** On the B1 recipe, group hold-out (no base gadget on both sides) drops accuracy 4.9 pp and macro-F1 7.9 pp, and an optimisation-level hold-out (train O0–O2/Os, test O3) collapses macro-F1 from 94 to 39. These are the earlier dataset's numbers.

Two things to keep honest here. The group hold-out for the *locked* recipe (learned neutral features, adversarial head), retrained for five seeds, gives **89.8 ± 2.5%** accuracy and **82.7 ± 4.3** macro-F1 (`eval/group_holdout_locked`). That is 2.6 pp below the locked model's random-split accuracy (92.45%)—a smaller leakage penalty than the B1 recipe's 4.9 pp, and the number to cite for generalisation. Two caveats bound it: the random-split and group-hold-out test sets are different sets (the latter is the held-out groups of the length-matched training pool), so the 2.6 pp is indicative rather than a paired difference; and per-class recall on held-out groups ranges from 94% (BENIGN), 93% (Spectre V1), 92% (Spectre V4) down to 76% (Spectre RSB) and 71% (INCEPTION), with every class retaining real support, so the headline does not rest on one class. The optimisation-level hold-out *cannot* be reproduced on the locked model: its length-matched training data carries no optimisation-level tag (verified: 18,897 of 19,053 records untagged, and the few that are tagged are O0/O1 only, with no O3 to withhold), so the 94→39 figure stands only for the earlier dataset and would need that data rebuilt with optimisation tags to re-measure. We report it as historical rather than as a property of the locked model.

2) Opcode shortcuts:

The shortcut is load-bearing: Replacing each class's trigger opcodes with `nop` (same length, 442 of 1,670 records altered) drops the flat hand-58 model from 95.15% to 78.44%. MDS falls from 97.8% to 0%, V4 from 99.2% to 0%, and L1TF from 67.6% to 0%. BENIGN and Spectre V1, whose rules are structural, do not move. For these classes the flat model *is* the hand rule.

Removing it: Training with trigger-masked augmentation closes the gap between intact and masked inputs from 27.4 pp to 1.1 pp. Masked macro-F1 rises from 0.503 to 0.805, and masked calibration error falls from 0.102 to 0.027.

Hand features as an x86 crutch: With the 58 inline features removed (5 seeds), macro-F1 improves on every axis: locked $0.813 \rightarrow 0.869$, ARM64 subset $0.618 \rightarrow 0.752$, x86 subset $0.897 \rightarrow 0.910$, and trigger-masked $0.803 \rightarrow 0.859$. A domain-adversarial (DANN) architecture head *hurt* in that setting (ARM64 $0.752 \rightarrow 0.456$) and was retired there. The locked model's adversarial head is a different configuration (neutral learned features), and it was selected on held-out RISC-V.

3) Learned vs. hand features (negative result): A ten-seed paired comparison on current code (learned node features vs. hand, full GINE) finds the learned features significantly *worse*: -1.66 pp accuracy ($p = 0.001$), -3.47 pp macro-F1 ($p = 0.008$), and Spectre V2 recall -9.55 pp ($p = 0.029$). An earlier per-class L1TF lift of +11.9 pp did not replicate; with 10 seeds the lift is 0.00 pp. A pre-registered TOST equivalence test [26] also rejects equivalence (-2.58 pp, 90% CI $[-3.81, -1.35]$). Spec-generic flat features beat hand-58 on the locked split (+1.88 pp) but lose under group hold-out (-1.55 pp, CI $[-2.01, -1.08]$); the locked-split win is an artifact. The value of the specification is portability (no ISA literals in code), not accuracy.

4) Cross-ISA transfer:

The length confound: The training BENIGN class consisted of 25–30-instruction ARM windows, while RISC-V benign test code is whole functions, so the model learned "long ⇒ attack". A model retrained on current code and scored on held-out RISC-V had a benign false-positive rate of 12% at ≤30 instructions, 1% at 31–60, 20% at 61–120, and 89% above 120. The same gap hit x86: with no x86 benign training data, 98.4% of real x86 benign functions were flagged as attacks (ARM: 27.4%). Length-matched training data and real x86/ARM benign code fix the cause. Windowed inference alone lowers RISC-V benign false positives from 28.6% to 3.0%, but also lowers detection from 74.8% to 54.8%.

Locked model on held-out RISC-V: On `riscv_loio_corpus_v2` ($n = 248$, 27 attacks; cluster-bootstrap 95% CIs) the benign false-positive rate is 2.3% [0.4, 4.8] and attack detection 48.1% [17.9, 77.1], Youden's $J = 0.46$. Per-class recall is BHI 0.75, RSB 0.33, V1 0.25 and V4 0.00 (store bypass does not transfer across ISAs); Retbleed, Inception, L1TF and MDS are unscored for lack of valid RISC-V examples. Canonical learned features (no hand features) fail to transfer ($J = -0.06$), probably because RISC-V's three-operand compare-and-branch is out of vocabulary, which hides the V1 branch itself.

With RISC-V training data: Mixing the RISC-V corpus into training (group-stratified) raises held-out RISC-V accu-

racy from $24.85 \pm 7.36\%$ to $75.45 \pm 13.20\%$ (5 seeds), with no x86/ARM regression ($95.89 \pm 1.69\%$ vs. $94.83 \pm 1.50\%$). Spectre V1 and V4 remain unmeasurable on that split for lack of data. The corpus is transliterated (Section IV-A3), so this measures adaptation to RISC-V surface form, not idiomatic RISC-V attacks.

Real RISC-V PoCs: The locked ensemble scores 0/11 on the 11 hardware-confirmed RISC-V PoC records (from published BOOM and T-Head C910 work), flagging only 1 as any attack: Spectre V1 0/7, Spectre V2 0/2, Spectre RSB 0/2. This must *not* be read as refuting the earlier transfer number: on the classes these PoCs cover, the same checkpoint already scored 0 recall on the transliterated corpus (V2, RSB), so the real set is consistent with what was measured there, not a new failure. What is genuinely new is Spectre V1 at 0/7, which the corpus could not measure at all. The honest statement is that the detector has *no* validated RISC-V attack-detection ability on independent data; the RISC-V contribution is cross-ISA adaptation of surface form and the length-confound analysis, not real-silicon RISC-V detection.

5) Synthetic-to-real transfer (hardware gadgets): Results are over 5 seeds, on the 173 held-out real violations (Section IV-A4).

Synthetic training does not transfer: A detector trained only on synthetic and compiled data labels most real gadgets Spectre V1, whatever their class. For example, it calls 74% of real MDS, 56% of L1TF and 73% of real V4 gadgets V1.

Per-class real-data models learn style: A model given real Revizor programs of only its own class labels real programs of *every* class as its own: "Revizor-generated $\Rightarrow$ my class". These models' 1.00 recall is withdrawn.

Joint real-data model vs. an opcode bar: One model trained on all four classes' real gadgets separates them on held-out data at 0.97–1.00. A standard-scaled bag-of-opcodes logistic regression gets 0.97 (L1TF 0.98, V1 0.93): each Revizor configuration's instruction set gives the class away. The graph model adds about +0.01 (L1TF) and +0.05 (V1). An earlier "+0.25–0.30 over opcodes" came from an unscaled baseline and is withdrawn. Class identity and Revizor configuration stay confounded until the classes are collected under a shared instruction set (**[TODO: shared-config Revizor campaign]**).

6) External check: a published kernel gadget: We reconstructed the primary case-study gadget of Kasper [27] (`find_keyring_by_name`, its PHT-MASSAGE-MDS label) in x86-64 from the paper's listings and ran it through our locked detector. It is flagged as an attack at 0.996 and classed SPECTRE_V1—consistent rather than contradictory, since our taxonomy names the speculation primitive (the PHT loop-exit misprediction) while Kasper names the leak channel (MDS). Removing the in-gadget transmit barely moves the score (0.979), i.e. the detector keys on the control/access structure and is nearly blind to the channel, which is exactly the MDS/port-contention blind spot our own dataflow features have.

C. RQ3: What Does Hardware Say About Mitigations?

1) Replay verification: Fuzzing flags a violation once; we separately check that each recorded violation *re-triggers* by replaying its exact saved inputs (`rvzr reproduce -i`, 200 inputs per directory, not regenerated from the seed—`rvzr minimize` regenerates and cannot replay, so it bails on most directories). On the i5-8300H the recorded L1TF (6/6), Spectre V1 (3/3) and Spectre V4 (14/14, both SSBP masks) violations re-trigger at Effective classification 100% after slow-path priming at a 500-input sample, confirming them as stable hardware leaks. The recorded **MDS** violations do *not* reproduce (0/6): each run is discarded on the fast path in $\sim$1.5 s with no slow-path signal, under SMT on and off alike. MDS is therefore the one class whose *recorded* violations we cannot confirm by replay. Fresh contract-fuzzing, however, re-finds an MDS violation within $\approx$15 test cases under SMT on *and* off (`rvzr_runs/MDS_REFUZZ_RESULTS.md`), so the MDS channel is live on this part and the 0/6 is recorded-input fragility, not a closed channel—we claim a verified MDS *channel* by fresh fuzzing, not a verified replay of the specific recorded gadgets; the replay drift is consistent with MDS's fill-buffer channel depending on transient microarchitectural state that the recorded inputs no longer recreate.

2) Fence-placement labels on silicon: Training a model to recognise *mitigated* code requires knowing that a fence mitigates. Our first "mitigated twins" were structural assumptions (Section V-C3), so we replaced them with hardware labels.

Protocol: For each held-out violation we write fenced variants of the original `program.asm`, keeping labels intact (`oracle/revizor/scripts/hw_label_variants.py`):

- `original` (positive control);
- `fence_all` (an `lfence` after every instruction);
- `twin` (the class boundary: V1 at the start of the jcc's taken block; V4 after each store; MDS/L1TF before each load);
- `shifted` (V1 before the jcc; V4 before each store);
- `after_load` (MDS/L1TF);
- `v1_fallthrough` (the jcc's exit path);
- `entry` and `tail`, with the same number of fences as the twin.

Each variant is re-run with `rvzr reproduce` three times on the violation's own inputs and configuration, with Revizor's speculation and observation filters turned *off*: a filtered test case would read as "no violation" and be mistaken for "mitigated". A variant is *vulnerable* if it violates in 3/3 runs and *mitigated* if in 0/3; anything else is flaky. A violation directory whose unmodified original does not reproduce 3/3 is unstable and contributes no labels. Revizor's parser forbids an instruction directly after a jump, so such placements are skipped. We checked that every program-level variant converts to exactly the sequence-level training variant, apart from jump offsets, which shift by the bytes the fences add.

Results: Table IV (3,024 reproduce calls; 806 labelled records).

- **A fence at the class boundary mitigates every class**, in every stable directory. For V4 this is the first evidence that

TABLE IV
HARDWARE LABELS FOR FENCED VARIANTS (i5-8300H, `rvzr reproduce` ×3, FILTERS OFF). CELLS: VULNERABLE / STABLE DIRECTORIES; 0 MEANS MITIGATED EVERYWHERE.

Variant	L1TF	MDS	V1	V4
stable / total dirs	63/64	33/33	45/54	21/22
twin (boundary fence)	0/63	0/33	0/41	0/21
fence_all	0/63	0/33	0/45	0/21
shifted (before)	—	—	0/41	0/21
after_load	0/63	0/33	—	—
entry	63/63	32/33	31/41	17/21
tail	62/63	32/33	31/41	21/21
v1_fallthrough	—	—	32/41	—

an `lfence`, and not only the SSBD MSR, removes these leaks.

- **Fences far from the leak do not mitigate**: entry/tail variants stay vulnerable in 289/316 stable cases.
- **The jcc's exit path does not mitigate V1**: 32/32 non-flaky directories stay vulnerable.
- **A fence just *before* the jcc or store also mitigates** (41/41 and 21/21). A plausible mechanism, not yet tested: in Revizor's programs the branch condition and the store address come from loads, and a fence waits for those loads, so the branch resolves (or the store address is known) before anything can speculate past it.

Stability: 9 of 54 V1 originals do not reproduce 3/3, and V1 misplaced variants are flaky in 7–10 of 41 directories. MDS, L1TF and V4 are clean.

Independent replay check: As a second, simpler confirmation that does not go through the variant generator, we replayed each recorded violation that reproduces on this box against a single `fence_all` copy of *its own* program, on the same recorded inputs (`oracle/revizor/scripts/reproduce_fenced_matrix.sh`; SMT on, matching the baseline matrix). The pairing is strict—baseline must leak (`violations=1`) and the fenced copy must not (`violations=0`) on the identical program and inputs. All **20/20** reproducing directories mitigate: L1TF 3/3, Spectre V1 3/3, and Spectre V4 14/14 across both SSBP masks (`4444444`, `5555555`); source `rvzr_runs/reproduce_fenced_summary.tsv`. This independently corroborates the two load-bearing results above—that an `lfence` closes the channel on silicon for every reproducing class, V4 included—on the unmodified recorded gadgets rather than synthesised variants. MDS does not appear here for the same reason it is absent from Table IV: its recorded violations do not reproduce on this part (0/6), so there is no baseline leak to mitigate. The `fence_all` placement is maximal, so this confirms the channel *can* be closed, not that any single targeted fence suffices—that is what the placement study above measures.

Mid-sequence counterexamples (in progress): In this variant set every non-mitigating fence sits at the start or end of the sequence, and every mitigating one sits inside it. A non-learned rule, "BENIGN iff some `lfence` is inside the sequence", therefore matches 799/806 labels (0.991). The set cannot tell structural understanding from fence position. We added four counterfactual variants that put a fence *inside* the sequence where it may not mitigate:

- `cf_wrong_path` (V1): `jcc; .bb_new: lfence; jmp`. In AT&T form this is exactly our pre-October V1 twin.
- `cf_nonleak`: a fence next to memory operations that `rvzr minimize` found unnecessary for the violation.
- `cf_after_bypass` (V4): a fence after the load that follows the essential store.
- `cf_fencepass`: Revizor's fence-insertion pass, which keeps every fence the violation survives.

[TODO: hardware labels for these variants on the held-out and training pools; fill Table IV rows and the interior-fence bar.]

3) Mitigation awareness against hardware labels:

A detector not trained on mitigations has none: Before any mitigation-aware model, we score the plain B1 detector (Section V-B; no fence labels in its training) on the hardware-labelled variants by whether its leak/benign call *flips* between a base gadget's leaking and mitigated arms—the measurement that isolates fence sensitivity, since the two arms share body and sandboxing and differ only in fence placement. Over five reproducing seeds (each gate-checked to reproduce its own locked-test accuracy on current code) the detector separates the two arms for only $\mathbf{0.20 \pm 0.14}$ of the 154 base gadgets—*below* a fence-count baseline (0.351)—and achieves strict separation (every leaking arm flagged *and* every mitigated arm cleared) for **0/154 in every seed**. The result is seed-dependent (0.00 to 0.40): the best seed clears the fence-count baseline only by flagging the many-fence `fence_all` variant, still calling the single-fence `twin` and `shifted` mitigations leaky; the worst never separates. The detector flags fenced and unfenced gadgets alike—it keys on gadget shape, not on whether a fence neutralises the leak. This is the gap a mitigation-aware model must close, and the motivation for training on these labels directly (leak-conditioning, Section V-C2).

Two models were trained with *structural* fenced twins:

- **allhw**: twins labelled BENIGN.
- **allhw2**: twins labelled BENIGN, plus misplaced-fence variants labelled as attacks. Some of those labels later proved wrong (Section V-C2): the old V1 twin fenced the exit path, and the shifted placements mitigate.

Table V scores both models against the hardware labels.

- allhw learned "has an `lfence` ⇒ safe".
- allhw2 reproduces its training labels exactly, including the wrong ones. It fails only on the fixed V1 twin (0.00) and the shifted variants (V1 0.00, V4 0.01); the bottleneck was label correctness, not model capacity.
- No model beats either non-learned rule. The interior-fence rule matches 0.991 of the labels because of how the current variant set is built (Section V-C2).

The next model (allhw3) trains on hardware labels only, including the mid-sequence counterexamples. A structure claim

TABLE V

ACCURACY AGAINST HARDWARE LABELS (806 FENCED VARIANTS OF HELD-OUT REAL VIOLATIONS; 321 VULNERABLE, 485 MITIGATED). THE TWO RULES USE NO LEARNING. MODELS: MEAN OVER 5 SEEDS.

Scorer	All	Vuln.	Mitig.
Interior-fence rule	**0.991**	1.000	0.986
Adjacent-fence rule	0.871	0.835	0.895
allhw2 (structural + misplaced)	0.851	0.996	0.755
allhw (structural twins)	0.619	0.083	0.974
Synthetic-only	0.154	0.354	0.022
allhw3 (hardware labels)	**[TBD]**	**[TBD]**	**[TBD]**

requires it to beat the interior-fence rule on those counterexamples. **[TODO: allhw3 result vs. the opcode, adjacency and interior-fence bars.]**

D. RQ4: Can the Generator Produce Oracle-Verified Leaks?

1) Syntactic validity: Initially only 2.3% of generated sequences were fully valid (71.1% of instructions). The first oracle-splice run gave 3 leaks, 4 safe and 73 unrunnable out of 80. A triage by assembler diagnostic found realiser bugs, fixed without retraining:

- **An unimplemented `<fn>` placeholder**: per-instruction validity 71.1% → 81.2%.
- **Register-width mismatches** (64-bit registers paired with `l`/`w`/`b` suffixes): x86 per-instruction 82.9% → 94.5%, and per-sequence validity 3.5% → 21.5%.
- **Cross-ISA mnemonic leakage**: masking opcodes that the target ISA's spec rejects gives 24.2% per sequence and 95.7% per instruction on x86.

The remaining failures are `.L` label tokens used as data operands and residual operand-type errors.

2) Per-ISA architectural validity: Because every leak oracle here is x86, the ARM64 and riscv64 arms of the generator need a quality measure that does not depend on an oracle. We use two: whether the realised sequence *assembles* as a unit (`llvm-mc`), and whether it *executes* to completion in a CPU emulator (Unicorn; no speculation, caches or branch prediction, so this is a necessary condition for a leak, never a leak claim). Together they separate "the generator cannot write valid code for this ISA" from "we have no oracle for this ISA". The measure first exposed a realiser bug: the inline assembler check that drops ill-formed instructions during realisation fired only for x86, so malformed ARM64 lines survived and only 28% of ARM64 sequences assembled as a unit, against 99% for x86. Extending the check to ARM64—a per-architecture `clang` invocation, activated only where the backend is present and otherwise failing open—raises ARM64 to x86 parity with no retraining: syntactic validity 28%→99% and emulator executability 25%→81% (Table VI), the latter now above x86's own 69%. The lift is broad across classes; the worst cases move the most—Spectre V4 (3% → 100% assembled) and L1TF/MDS (10% → 95–100%). A residual 19% of ARM64 sequences assemble but fault in the emulator, concentrated in INCEPTION (38% executed) and Spectre V2 (62%): a semantic-realiser gap, not a syntactic one. This is a syntactic/executability result on two ISAs; it is not a leak measurement, and ARM64 still has no leak oracle.

TABLE VI

GENERATOR OUTPUT VALIDITY PER ISA, BEFORE AND AFTER EXTENDING THE REALISER'S INLINE ASSEMBLER CHECK TO ARM64 (`generator.pt`, 40 SAMPLES/CLASS, SAMPLING MASK OFF TO ISOLATE THE REALISER). *ASSEMBLES*: `llvm-mc` ASSEMBLES THE SEQUENCE AS A UNIT; *EXECUTES*: UNICORN RUNS IT TO COMPLETION (NO SPECULATION—A NECESSARY CONDITION FOR A LEAK, NOT A LEAK CLAIM). X86 IS THE UNCHANGED REFERENCE; ITS SMALL MOVEMENT IS SAMPLING NOISE.

	assembles		executes	
ISA	before	after	before	after
x86_64	0.99	1.00	0.62	0.69
ARM64	0.28	**0.99**	0.25	**0.81**

TABLE VII

GENERATOR OUTPUT, PER-ISA ARCHITECTURAL VALIDITY (RATES OVER SAMPLED CANDIDATES). **NOT A LEAK MEASUREMENT:** "ASSEMBLES" AND "EXECUTES" ARE NECESSARY CONDITIONS FOR A LEAK, NOTHING MORE. THE MASK CONDITIONS ARE THE SAMPLING RULE OF SECTION A-C: NONE, THE SPECIFICATION RULE, AND THE SPECIFICATION RULE PLUS AN ASSEMBLER CHECK.

ISA	mask	realised	assembles	executes
x86_64	none	0.99	0.99	0.62
x86_64	specification	0.99	0.99	0.65
x86_64	+ assembler	1.00	1.00	0.52
arm64	none	1.00	**0.15**	0.25
arm64	specification	1.00	0.77	0.72
arm64	+ assembler	0.99	**0.95**	0.84
riscv64	+ assembler	1.00	**1.00**	0.02

With the sampling mask of Appendix A-C the non-x86 arms improve further, and riscv64 can be sampled at all for the first time (Table VII); in each case the cause was mechanical and fixable (Appendix A).

The riscv64 row is from a generator retrained with the riscv corpus; it is the first riscv64 generation this project has. Its *executes* rate needs a two-stage reading, because the two emulation backends fail for different reasons. Under Unicorn (the same backend as the x86/ARM rows) riscv64 executes at 0.02, but that is an *executor* limit, not a generator one: Unicorn models roughly the RV64I base and raises an illegal-instruction fault on the M/A/C/Zicbom forms the RV64GC generator freely emits, so $527/540$ sampled bodies fail at instruction decode and 0 reach a memory access. Rerunning the identical protocol through a QEMU user-mode RV64GC backend (`gen/precheck.py:RiscvQemuRunner`) removes that wall entirely— *zero* decode faults—confirming the bodies are valid RV64GC. What it exposes is the next layer: $530/540$ now fault on *memory* and only $\sim$1% run clean. Two interventions isolate the cause and, in doing so, show the *executes* rate is ill-posed for this generator rather than merely low. We pinned every memory-operand base to a reserved register the body never clobbers (so a load cannot leave the seeded sandbox) and separately gave the QEMU harness demand paging (map the faulting page and retry, as the Unicorn path does): both leave the $530/540$ memory-

fault count *byte-identical*. So the faults are not clobbered load bases and not RW-demand-pageable. What remains is control-flow: the realiser retargets direct branches to a safe label but leaves `jalr/ret` indirect transfers untouched (for the return-speculation classes the indirect branch *is* the gadget), and every register is seeded into a non-executable data sandbox, so an indirect transfer fetches from non-executable memory and faults. The x86 and ARM figures are not a clean baseline for this: Unicorn maps its sandbox executable, and an x86 jump into zeroed sandbox memory decodes as a benign `add [mem]`, so it coasts to the instruction budget and scores as "executes", whereas the all-zero RV64 encoding is an illegal instruction, so the identical control-flow pattern faults. The riscv bodies are valid RV64GC (zero decode faults); what a bare user-mode emulator cannot score is a return/indirect-branch gadget without a harness that supplies a valid branch target—which is exactly what a real leak oracle, not a runnability check, would provide. **[TODO: the riscv executes rate is gated by return/indirect-branch targets, not decode (removed) or load addresses or paging (both shown no-op); a comparable number needs a harness that seeds valid branch targets, i.e. the riscv leak oracle, not a realiser or emulator tweak.]**

3) Yield: On x86 the loop drives the confirmed-leak yield to about 1.0 for Spectre V1 and 0.98 for Spectre V4; the control-flow validity that unlocks V1 and the length conditioning plus dataflow-preserving splice that unlock V4 are detailed in Sec. V-D4 (Table VIII). V1 reaches its ceiling at round 0 (about $0.50 \rightarrow 0.97$ in one round, then flat across seeds); V4 climbs over the loop. Auditing confirms genuine discovery rather than mode collapse: about 1,600 distinct leaking gadgets across runs, median pairwise similarity 0.32. The base multi-class loop, the self-supervised-pretraining diversity/yield trade, and the length-diversity audit are in Appendix A.

4) Control-flow validity and length control raise adjudicability: Cold generations are mostly *unadjudicable*: sampling the generator and asking Spectector to rule gives only **6%** adjudicable (112 of 119 UNRUNNABLE, zero leaks). Inspecting the failures shows the cause is control flow the oracle cannot model or the victim splice cannot host: a `ret` in the body returns before the spliced secret→transmit sink, so the sink is unreachable, and a label-free branch given a self-relative target (`.+2`) jumps into the middle of the next instruction. Three changes, all at sampling/realisation time and none retraining the reward, remove this class of failure:

1) a class-aware decode mask that forbids the stranding control flow (unconditional direct jumps; indirect transfers for the conditional-branch classes) while keeping each class's own mechanism (the indirect call for Spectre V2, the `ret` for Retbleed);
2) a realiser rule that retargets a self-relative branch to a forward label at the end of the gadget—assemblable, a plain forward conditional branch the model's PHT semantics adjudicate, and leaving the sink reachable;
3) stripping a body `ret` at splice time (rather than masking it at decode, which removed the model's stop cue and made every gadget run to the length limit) for every class whose mechanism is not itself a return.

TABLE VIII
ORACLE ADJUDICABILITY AND CONFIRMED-LEAK YIELD, COLD VS. THE FINAL CONFIGURATION: CONTROL-FLOW FIXES, A LENGTH-CONDITIONING TOKEN SET TO `long`, AND A DATAFLOW-PRESERVING SPLICE FOR STORE BYPASS (CLUSTER SPECTECTOR, X86). YIELD IS THE FRACTION OF REALISED SAMPLES THE ORACLE CONFIRMS LEAK. PER-ROUND DEVELOPMENT IS IN APPENDIX A.

Configuration	Class	Adjudicable	Leak yield
cold (no fixes)	V1	6%	∼0
final (fixes + len-cond, `long`)	V1	98%	**1.00**
	V4	97%	**0.98**

With these in place we add a length-conditioning token (`<LEN_short|med|long>`), so a target length can be requested at sampling time; this both keeps gadgets in the oracle-adjudicable band and makes the length–yield dependence controllable. On the cluster oracle, across five rejection-sampling rounds (k=60), the effect is large (Table VIII).

Cold, the generator is almost entirely unadjudicable (6%). The control-flow fixes alone raise V1 adjudicability to 98% and its confirmed-leak yield to ∼1.0 with no reward retraining—so for V1 the gain is the validity of the generated control flow, not the loop. Spectre V4 needs more: its leaks concentrate at long lengths (its confirmed leaks average 52 instructions, because the store-bypassed value must survive a wide speculation window), and the generated body must actually carry the stale value to the transmit sink. With the length token set to `long` and a splice that reserves the store-bypassed value in a register the body cannot clobber and forces it into the transmit, the loop (k=120) raises V4 confirmed-leak yield monotonically from 0.04 to **0.98** over eight rounds—a 2.4× lift over the 0.40 ceiling reached without the dataflow-preserving splice. V4 is thus the one class where the oracle-in-the-loop concentration is visibly doing work, whereas V1 is already saturated at round 0. At scale this run produced 936 confirmed V1 leaks and 495 confirmed V4 leaks (of 960 realised samples each), with no V1 SAFE verdicts; the earlier `med` setting is retained only to harvest a large V4 SAFE set (negatives for the detector and the filter). Extending the token set with a fourth bucket (`xlong`, >32 instructions) and retraining shows the dataflow is not length-fragile: at `xlong` the V4 confirmed-leak yield climbs 0.80→0.95 over five rounds (V1 holds ∼0.87), so pushing gadgets past the `long` band does not break the store→stale-load→transmit chain—the splice keeps the stale value alive across the wider window rather than losing it. A crude length alternative—capping the token budget—does the opposite: it truncates gadgets mid-body and drops adjudicability to 0%, so the *token*, which emits complete gadgets of the target length, is the mechanism that works.

5) Steering leaking vs. mitigated generation: The 821**[TODO: reconcile with the 806 labelled records of Table IV]** labelled variants are not only a detector test set; they are the training signal for the leak-conditioning token of Section III-D. With the token trained on them

(leaking arm LEAK, mitigated twin SAFE, class preserved; held out by base), conditioning moves generation as intended. Structurally, across all four hardware classes the LEAK arm is fence-sparse and the SAFE arm heavily fenced (mean fence tokens per sequence, leak→safe: V1 $0.4 \to 9.0$, V4 $1.7 \to 18$, MDS $3.4 \to 22$, L1TF $3.0 \to 32$; opcode Jensen–Shannon 0.15–0.29; the sample sets do not overlap)—a structural readout, not a leak claim. Adjudicated by Spectector, the token controls whether a Spectre V4 gadget actually leaks: the LEAK arm leaks in 0.80 of adjudicable gadgets (16/20, Wilson 0.58–0.92) against 0.04 for SAFE (2/49, 0.01–0.14), with non-overlapping intervals—the first oracle-grounded control over mitigated-versus-unmitigated generation here. Spectre V1 is inconclusive by construction: its Spectector victim template carries the bounds-check bypass itself, so the spliced body and its fences do not gate the leak and both arms leak ≈ 0.95; V4's template routes the leak through the body, so body fences gate it. The result is *per adjudicable gadget*. Raw discovery yield (confirmed leaks per sample drawn) rises only $1.7\times$ for V4 (0.19 vs 0.11, overlapping intervals at $n{=}80$) because roughly two-thirds of fence-sparse LEAK gadgets carry at least one instruction outside Spectector's model (the `bt/bsf/cmov` forms common in real Revizor gadgets) and a single unmodelled instruction voids the verdict—an oracle-coverage limit, not a property of the token.

Recovering net yield with a sound guard: Every unmodelled instruction we observed on these LEAK arms is speculation-inert—`cmov`, `bt/btc/btr/bts`, `bsf/bsr`, `bswap`, `div/idiv`, `rdtsc`—none of them a speculation barrier, so Spectector passing through them cannot hide a fence. The only residual risk is the opposite one: by ignoring such an instruction Spectector could *over*-report a leak that the ignored instruction would in fact have suppressed. We therefore gate the reclaim with a strip-and-rerun differential: a DATA/CONTROL verdict with `unsupported_ins` > 0 is accepted as LEAK only if (a) every unsupported mnemonic is in the known-inert set—else the gadget is refused, as the op could be an unmodelled barrier—and (b) the gadget with all unsupported-mnemonic instructions deleted still leaks under the fully modelled oracle. Validated on 60 LEAK draws per class with positive and fenced controls passing (the canonical leaking template is LEAK, its fenced form SAFE), the differential never had to reject a leak over all 40 reclaim candidates (the ignored inert ops never carried the leak), while the safety catch refused six draws whose unsupported ops—`cwtd`, `setno`, and a realiser-malformed three-operand `add`—were not provably inert. Net yield, measured per sample drawn, then rises from 0.18 to 0.42 for V4 ($2.3\times$) and from 0.45 to 0.88 for V1, at zero admitted false positives. The guard is wired into the generate–adjudicate loop behind the `--netyield-guard` flag (a validator wrapper that reclaims through the identical, validated differential); the conservative discard stays the default oracle behaviour, and we report the recovered rate as the achievable net yield when the relax is enabled.

6) Spectre V2 under Spectector-Combined:

V2 is now adjudicable at a rate comparable to V1: Measured on the cluster over 2,830 generated SPECTRE_V2 bodies (the local container OOMs on heavyweight victims; Table IX), Spectector-Combined rules on **76.6%** of them—comparable to V1's 82% and V4's 87% under upstream Spectector, and a step change from the ∼15% upstream manages on V2. That is the coverage this whole thread was for: V2 moves from no working oracle to symbolically adjudicable.

Two controls keep the number honest, and one of them already did its job. A *positive-control anchor*—a known-leaking V2 gadget that must return LEAK unfenced and SAFE fenced—is run in every shard; it is the guard that a SAFE verdict is a real analysis result and not a silent timeout. At a 50-instruction window the anchor passes in all ten shards; at a 200-instruction window it *fails in all ten* (the wider analysis times the anchor out), so that run's numbers are discarded rather than reported — the same timeout-masquerading-as-safe failure that produced this project's retracted V4 result, caught here by the control before it reached a table. The 76.6% is the anchor-validated window, and the anchor passed in all ten shards of this run.

The generated V2 bodies leak rarely even when adjudicable: 26 of 2,168 (1.2%), which de-duplicate to **16 distinct** leaking gadgets. A landing-pad fence (the second control) turns 15 of the 16 SAFE; a single distinct gadget leaks *through* the fence in all three of its draws—a V2 body whose speculative transmit the one landing-pad fence does not dominate, which we flag rather than suppress.[1] So the class-conditioned generator emits V2-shaped code that mostly does not actually leak under the V2 contract, consistent with the earlier near-zero V2 rejection-sampling yield—the gap there was partly the oracle (now closed) and partly the generator, which the end-to-end bootstrap below now isolates as the binding one.

The generator, not the oracle, is the V2 bottleneck: a negative result: With the oracle working we ran the oracle-labelled bootstrap (*generate* → `-v 2` *label* → *keep leaks* → *fine-tune*) end to end on the cluster. Two clean runs reproduce the adjudicability but confirm that synthesis does not supply positives. A 1,200-body run (eight rounds of 150) ruled on **998 bodies (83.2%)**—in the same 77–83% adjudicability band—yet yielded **one** oracle-confirmed leak (0.10% of adjudicable); a prior 300-body run yielded none. Across roughly 2,000 adjudicated bodies in total the generator produces on the order of five confirmed V2 leaks. The single bootstrap leak is a genuine V2 gadget—an indirect call (`callq *%rax`) whose mispredicted target speculatively executes a secret-dependent load—but one positive cannot define a leak-versus-safe filter, and a sub-1% yield cannot bootstrap a training set. We therefore report this as a negative result: oracle-labelled self-training does not manufacture SPECTRE_V2 positives at useful scale with this generator. Spectector-Combined's role for V2 settles accordingly as a *labeler*—it confirms the rare positive and supplies a large SAFE set (∼1,000 bodies)—rather

[1] `source_id` hash `7944e898`; the earlier 590-body run reported zero such anomalies, so this is a property of the larger sample, not a regression.

TABLE IX

SPECTRE_V2 ADJUDICABILITY UNDER SPECTECTOR-COMBINED (`-v 2`), CLUSTER. THE 50-INSTRUCTION WINDOW IS THE ANCHOR-VALIDATED ONE, NOW MEASURED OVER 2,830 BODIES; THE 200-INSTRUCTION WINDOW'S POSITIVE-CONTROL ANCHOR TIMED OUT IN ALL TEN SHARDS OF AN EARLIER 590-BODY PROBE, SO IT IS SHOWN ONLY TO RECORD THAT IT WAS REJECTED. ADJUD. = ADJUDICABLE; LEAK/ADJ. = LEAKING BODIES AS A SHARE OF ADJUDICABLE (INSTANCES; THE W=50 LEAKS DE-DUPLICATE TO 16 DISTINCT GADGETS, 15 FENCE-MITIGATED).

window	bodies	adjud.	leak/adj.	fence→safe	anchor
w=50	2,830	**76.6%**	1.2% (26)	23/26	passes
w=200	590	77.8%	1.1% (5)	5/5	**fails**

than a yield engine, so scalable V2 positives must come from the hardware campaign (Sec. IV-A4), not synthesis.

E. RQ5: Can We Spend Fewer Oracle Calls?

1) The classifier is not a usable filter: Two findings came from checking the classifier against the oracle data.

- **V1 has no SAFE class.** Generated Spectre V1 samples contain 1,531 leaks, 0 safe and 456 unrunnable. An earlier "classifier vs. oracle AUC 0.44" therefore measured leak vs. *unrunnable*, not leak vs. safe.
- **On V4 the classifier is inverted.** Spectre V4 is the only class with both outcomes: 460 unique gadgets after de-duplication, 273 leak and 187 safe. Results:
  - the locked classifier's attack probability has AUC 0.14 (0.27 within a round) and correlates -0.66 with sequence length;
  - sequence length alone gives 0.75 (0.59 within round);
  - an opcode-bag logistic regression, evaluated leaving one run out, gives 0.44.

The loop round confounds everything here: the V4 leak rate rises from 0.18 to 0.85 across rounds, so we report within-round AUC.

2) The non-learned gate:

Result: Table X measures each stage against the target we fixed before measuring: remove at least 80% of UNRUNNABLE while losing at most 2% of LEAK. **The target is not met.** The full gate removes 18.2% of unrunnable candidates for a 1.3% loss of leaks, raising the share of oracle calls that return a verdict from 0.574 to 0.618. The leak cost is well inside budget; the benefit is a quarter of what we asked for.

The diagnosis is specific. Stage B is accurate but narrow: it accounts for essentially all of the gain (17.4% of unrunnable at 0.8% of leaks), and what it rejects is a short list of instructions genuinely absent from Spectector's table—`endbr64` (241 occurrences), `rdtsc` (77), `clflush` (45), `syscall` (23), `cpuid`, `prefetch*`. Stage A removes nothing at all, which is the ceiling we predicted: the sidecars are *post-realiser* and the realiser already drops instructions that do not assemble, so stage A's measured contribution here is a lower bound on its value for raw generator output. Stage C removes 0.8% and is not worth its cost on this data.

TABLE X

PRE-ORACLE GATE OVER UNIQUE GENERATED GADGETS CARRYING A SPECTECTOR VERDICT. TARGET FIXED BEFORE MEASUREMENT: $\geq$80% OF UNRUNNABLE REMOVED, $\leq$2% OF LEAK LOST; THE TARGET IS NOT MET. n = 3,921 UNIQUE SEQUENCES (1,974 LEAK, 277 SAFE, 1,670 UNRUNNABLE). WILSON INTERVALS IN THE SOURCE.

Rule	unrun. removed	leak lost	calls saved
A: assembles	0.000	0.000	0.000
B: oracle front end	0.174	0.008	0.082
C: emulates	0.008	0.006	0.006
A+B+C (full gate)	**0.182**	**0.013**	0.088

TABLE XI

SPECTRE_V4 LEAK-VS-SAFE DISCRIMINATION, 460 UNIQUE GENERATED GADGETS. WITHIN-ROUND AUC CONTROLS FOR THE LOOP-ROUND CONFOUNDER; PROSPECTIVE TRAINS ON ROUNDS $\leq r$ AND TESTS ON $r{+}1$. BOTH COLUMNS ARE AUC.

scorer	within-round	prospective
sequence length (bar)	0.587	—
opcode-bag LR	0.650	0.627
frozen-encoder head	**0.656**	**0.772**
pairwise (order-aware)	0.628	0.764

What this says is that 1,366 of the 1,670 unadjudicated candidates assemble, parse on their reachable path, and execute—so their unrunnability comes from something no static gate can see: a Spectector timeout, a translation failure after a successful parse (`x86_to_muasm.pl` can still throw `could_not_translate`), or an unsupported instruction on a path other than the straight-line prefix. Reaching 80% would require predicting those, which is a learned-filter problem, not a parsing one.

3) The learned filter:

Result: the retargeted filter clears the bar, modestly: On the 460 unique V4 leak/safe sequences, a logistic head on the frozen *embedding* (not the classifier's attack probability, which was inverted) separates leak from safe at within-round AUC **0.656**, above the sequence-length bar's 0.587 and level with an opcode-bag model's 0.650 (Table XI). Under the prospective split (train rounds $\leq r$, test $r{+}1$) the encoder head reaches 0.772, so the signal is not an artefact of round. The embedding carries usable leak signal where the classifier output did not—the fix was the target (binary leak, not trace length) and the representation (fused embedding, not the softmax). The gain over length is real but small, and the leaks-per-oracle-call headroom is limited precisely because the generator's own yield is already high by late rounds (the filter's value concentrates in early rounds and harder classes, as the headroom argument above predicts). The encoder built 360 of 460 PDGs; the other 100 are counted as unselectable, not dropped. An order-aware *pairwise* head (a RankNet-style loss over within-round leak/safe pairs on the same embedding) gave no improvement—within-round AUC 0.628 and prospective 0.764, at or below the pointwise head—limited by the 262:17 leak-to-safe imbalance, which starves the pairwise objective of negatives; we keep the pointwise head.

TABLE XII

ENCODER HEAD AS A PRE-ORACLE GATE INSIDE THE LOOP (CLUSTER SPECTECTOR, X86, k=120). "FILTERED" = CANDIDATES WITHHELD FROM THE ORACLE; "PER CALL" = CONFIRMED LEAKS PER ALLOWED ORACLE CALL. V1 HAS NO SAFE CLASS SO NOTHING IS FILTERED.

Class	Round	filtered/120	oracle calls	leaks per call
	0	98	22	**1.00**
	1	82	38	**1.00**
V4	2	55	65	0.97
	3	58	62	0.98
	4	40	80	**1.00**
V1	0–4	0	120	0.90–0.98

TABLE XIII

MINIMISATION OF REAL VIOLATIONS (REVIZOR INSTRUCTION-REMOVAL PASS, RECORDED-INPUT REPLAY). BODY = NON-INSTRUMENTATION INSTRUCTIONS IN THE MEASURED REGION; ESSENTIAL = THOSE THE PASS COULD NOT DELETE WHILE THE VIOLATION STILL REPRODUCED ON HARDWARE. DIRS = VIOLATION DIRS WHOSE PASS COMPLETED (STOCK `rvzr minimize` REGENERATES INPUTS AND REPRODUCED ONLY 4/173; REPLAYING EACH VIOLATION'S RECORDED INPUTS COMPLETES 151/173). SOURCE: `eval/min_report.md`.

Class	Dirs	Body (median)	Essential (median)	Reduction
L1TF	60/64	16.0	2.0	88%
MDS	26/33	21.0	10.5	49%
V1	44/54	18.0	5.5	69%
V4	21/22	47	14	70%

In the loop: the gate spends oracle calls almost only on leaks: We then wired the frozen-encoder head into the rejection-sampling loop as a pre-oracle gate: a candidate is sent to Spectector only if its leak score clears a threshold, and the saved call budget is measured directly as confirmed leaks per oracle call. The gate was trained on the earlier dataflow-splice run and applied to a fresh five-round run (generator `generator_lencond`, k=120), so the evaluation is prospective, not on its own training samples. On V4 the gate discards 98, 82, 55, 58 and 40 of the 120 candidates per round and sustains **1.00, 1.00, 0.97, 0.98, 1.00** confirmed leaks per allowed oracle call (Table XII)—at round 0 that is 22 calls for 22 leaks where the ungated yield was 0.18, a 5.5× reduction in wasted calls, narrowing to 1.5× by round 4 as the generator's own yield climbs, exactly the early-round head-room the argument above predicts. On V1 the gate filters nothing (every sample scores leaky, which is correct—V1 has no SAFE class), so its allowed-call yield equals the raw yield and the gate is a no-op, as intended. The gate keeps ∼100% precision on the calls it allows while cutting 33–82% of the oracle calls on the class that has anything to cut.

F. RQ6: Are Discovered Leaks Minimal?

For real violations Revizor's instruction-removal pass (Section III-G) gives the instructions each violation needs (Table XIII). On 151 of 173 hardware violations the pass completes, and real gadgets reduce substantially but not to a point: median body falls from 16–47 instructions to an essential core of 2 (L1TF), 5.5 (V1), 10.5 (MDS) and 14 (V4)—a 49–88% reduction, with the leak genuinely distributed across several instructions rather than the single scaffold instruction the generated-leak measurement below collapses to. Stock `rvzr minimize` reproduced only 4/173 (it regenerates inputs); replaying each violation's recorded inputs was required to drive the pass.

Generated leaks minimise to the scaffold, not the body: We ran the plan on generated leaks: 1-minimality delta debugging [21] with Spectector as the test, on 40 confirmed Spectre V1 and 40 confirmed Spectre V4 gadgets (original body medians 30 and 60 instructions). *Every one of the 80 minimises to a single instruction*, V4 included, even though its gadgets are twice as long because the store-bypassed value must survive a wide speculation window. This is a result about where the leak lives, not about the generator: the victim is a fixed splice scaffold (for V1 a bounds check, a secret load and a transmit; for V4 a store that is bypassed, a stale reload and a transmit through a reserved register) into which the generated body is inserted, and the generated body is an inessential speculation-window filler, so delta debugging correctly strips it to nothing and leaves the one scaffold instruction that carries the leak. That V4 behaves exactly like V1 is the sharper point: even the class whose mechanism we built a dataflow-preserving splice for turns out to carry its leak in the scaffold's reserved-register transmit, not in any learned instruction. Generated-gadget minimality as posed here therefore measures the minimality of our victim template, which is 1 by construction, rather than of the learned code. The informative version of the question—the smallest *complete* leaking program—requires delta-debugging the victim and body together, which we do for real violations with Revizor's pass (Table XIII) and leave as the correct form of the generated-leak measurement.

G. RQ7: Can a Driven Emulator Stand In for an Oracle off x86?

Validation against evidence it did not produce: A model is worth nothing on its own word, so the emulator is scored against the 806 hardware-labelled fenced variants (Section V-C2), whose labels are a measured property of a real CPU, and against Spectector's verdicts on generated x86 gadgets. Because it is executed rather than pattern-matched, the emulator runs each variant's committed Intel-syntax `program.asm`: the corpus's AT&T `sequence` field lost its branch labels in the assemble/disassemble round trip (they survive only as objdump annotations such as `jnp 0x7e <.bb_0.1>`, which no assembler accepts), so control flow has to come from the original program text (`oracle/revizor_asm.py`). Of the four hardware-labelled classes only Spectre V1 is within the modelled mechanism; the other three are reported separately rather than folded into an accuracy figure. **The result is negative, and the reason is structural.** The emulator returns SAFE for all 806 hardware-labelled variants: it finds not one speculative leak. Its apparent 0.580 agreement on the in-scope class is therefore an artefact

of 58% of those labels being BENIGN—it is a constant predictor, and the interior-fence rule beats it at 0.987 (Table XIV).

The cause is not a missing mechanism but a mismatch of threat model. Revizor's generated programs mask *every* address into their sandbox (`and reg, 0b1111111111111`), and that instrumentation is on the mispredicted path too, so such a program never reads outside its sandbox even speculatively. There is consequently no out-of-bounds secret for a secret-versus-public comparison to find, and the construction that makes the check work on a textbook Spectre gadget cannot express what Revizor's violations actually are: a difference in *which in-sandbox addresses* are touched under speculation, relative to what the contract permits. Detecting those requires Revizor's own method—many inputs, grouped into equivalence classes by the model's contract trace—rather than a secret/public split.

Re-posing as a contract check: better framing, still not validated: We then built exactly that (`check_contract`): run many random inputs, group them by their non-speculative trace, and flag a leak when forced speculation distinguishes inputs the contract calls equivalent. This is the framing the hardware labels were produced under, and it does apply to the address-masked programs the secret/public mode was blind to—only 13 of 120 Spectre V1 variants now go unadjudicated, against essentially all of them before. But it still does *not* reproduce the labels: agreement on the in-scope class is **0.61**, against the interior-fence rule's 0.98 (Table XIV). The per-variant breakdown says why, and it is informative rather than random: the emulator agrees with hardware on the boundary-fence twins (0.90) and on `fence_all` (0.88)—it correctly sees a fence in the taken block stop the forced speculation—but it misses the `shifted` mitigation (0.48; the fence sits *before* the branch, and real silicon mitigates because it serialises the loads feeding the branch condition, which a model that forces the branch regardless cannot represent) and it calls most genuinely vulnerable `entry`/`tail`/fall-through variants safe (0.39–0.44; forcing the other path produces no distinguishable observation for them). So the contract re-posing fixed the applicability problem and left a fidelity problem: the emulator's unconditional force-the-other-path model is too coarse to match what the predictor and the branch's data dependence actually do.

What the emulator is therefore good for, and not: It is *not* validated against silicon—neither the secret/public mode (0.58, a constant predictor) nor the contract mode (0.61) beats a positional rule—so no ARM64 or riscv64 leak verdict from it is hardware-grade, and the paper claims none. What survives is narrower: on textbook-shaped gadgets it gives the right answers on two ISAs, including the fence-placement discrimination the hardware labels confirm, and the indirect-branch and return mechanisms added for V2/RETBLEED are unit-tested on hand gadgets (Section VI-C). That makes it a usable sanity check on *generator* output of those shapes, and the contract-mode machinery is the right scaffold for a higher-fidelity predictor model, but it is not a substitute for the hardware oracle.

TABLE XIV

SPECULATIVE EMULATOR AGAINST GROUND TRUTH IT DID NOT PRODUCE. IN-SCOPE = THE CLASS WHOSE MECHANISM IT MODELS; HW = HARDWARE LABELS. INTERIOR/ADJACENT ARE THE NON-LEARNED BARS OF TABLE V. THE EMULATOR RETURNS SAFE FOR EVERY ONE OF THE 806 VARIANTS, SO ITS "AGREEMENT" IS JUST THE BENIGN BASE RATE AND IT LOSES TO BOTH BARS. † NOT A LIKE-FOR-LIKE COMPARISON: SPECTECTOR ADJUDICATES THE WHOLE SPLICED VICTIM PROGRAM, THE EMULATOR SEES ONLY THE GENERATED BODY, AND ALL 300 BODIES ARE BRANCH-FREE.

Reference	n	Emul.	Interior	Adjacent
HW, V1 (in scope)	224	0.580	**0.987**	0.786
HW, L1TF (out)	314	0.602	1.000	0.908
HW, MDS (out)	165	0.612	0.988	0.903
HW, V4 (out)	103	0.631	0.981	0.893
Spectector, gen. x86	300	n/a†	—	—

VI. DISCUSSION

A. Lessons

Measurement bugs look like results: An InvisiSpec harness that counted confident cache hits reported "V4 leaks 40/40" (it recovers 0/40 correct bytes); oracle timeouts read as SAFE; an oracle image invisible to the compute node read as "0% yield"; a label-conditioned filter applied to the test pool inflated accuracy. Each produced a plausible number. What caught them was positive controls in every run—an anchor that must leak unfenced and stop leaking fenced—and checking against an oracle the component under test did not produce.

Shortcuts are the default: Opcode mix, sequence length, the Revizor configuration's instruction set, fence presence and fence position have each explained an apparent result (Sections V-B2, V-B4, V-B5, V-C3). In every case a non-learned bar exposed it, which is why every learned number here sits next to one.

An oracle must match the programs it judges: A secret-versus-public comparison cannot express a Revizor violation whose programs never leave their sandbox (RQ7); Spectre V1's victim template leaks on both arms of the leak-conditioning token (RQ4); and generated leaks minimise into the fixed scaffold rather than the learned body (RQ6). Each time, the shape of the harness, not the generator, determined the answer.

Coverage, not data, bounds per-class claims: Which classes we can verify is set by which mechanisms some oracle models (Table II), not by how much training data exists.

B. Limitations and Threats to Validity

- **One CPU, one fuzzer.** Hardware results come from one Coffee Lake part and Revizor. BHI likely needs a newer Intel part, and Inception affects only AMD Zen.
- **x86-only hardware verification.** Spectector and our hardware are x86. ARM64 and riscv64 have only the emulator-driven check of Section III-E1, which models conditional-branch speculation alone, uses concrete inputs, and is a model: its agreement with silicon is measured on x86 and assumed, not shown, elsewhere.

- **An emulator cannot see a leak by itself.** Unicorn and QEMU model no speculation, so every emulator result here is either a *runnability* statement or the output of a speculation model we drive ourselves—and that model reproduces none of the 806 hardware labels, so it is not a substitute for the hardware oracle.
- **Threat models do not transfer between oracles.** A secret-versus-public comparison expresses a textbook Spectre gadget and cannot express a Revizor contract violation, whose programs never leave their sandbox. An oracle has to be matched to the shape of the programs it will judge.
- **Class–configuration confound** in the real corpus (Section V-B5).
- **Small held-out sets.** Real held-out sets have 22–64 gadgets per class. RISC-V attacks number 27 across a handful of families, with wide confidence intervals.
- **Shortcuts.** Length, opcode mix, fence presence and fence position have each explained an apparent result. Every reported number sits next to the relevant non-learned bar.
- **Variance.** Per-class recall varies by up to 15–20 pp across seeds for small classes. We report only multi-seed results.

C. Next Steps

Ordered by coverage gained per unit of effort, and separated by what each actually needs.

No new hardware:

1) **Hardware-label the mid-sequence counterexamples and the training pool**, then train the hardware-label-only detector and report it against all three non-learned bars (in progress on the i5).
2) **Raise the speculative emulator's fidelity.** The contract re-posing (Section V-G) fixed applicability but agrees with hardware at only 0.61, because forcing the other successor ignores the predictor and the branch's data dependence. Until a higher-fidelity model beats the positional rule, *no* mechanism added to the emulator can be validated against the one ground truth we hold, so this gates the next item.
3) **Validate the emulator's indirect-branch and return models.** Both are implemented and unit-tested on hand gadgets; they are the one route to V2, BHI, RETBLEED and INCEPTION that needs no hardware and covers all three ISAs, but every verdict stays model-level until item 2 makes validation possible.
4) **Extend Spectector-Combined to RETBLEED.** V2 is now 76.6% adjudicable through it (Section V-D6); settle whether its return model covers the return-buffer-underflow fallback before trusting a RETBLEED verdict.
5) **Learned filters beyond V4.** The V4 head is in the loop (Section V-E3); V1 needs near-miss negatives before a leak-vs-safe filter is even defined, and V2 needs more than its 16 distinct positives.
6) **Minimality**: tabulate the Revizor instruction-removal results already produced (Table XIII), and delta-debug victim and body together for generated leaks.
7) **A shared-instruction-set Revizor campaign**, to break the class–configuration confound that currently lets a bag-of-opcodes model match the graph model on real gadgets.

Needs hardware we do not have: The machine used here (Coffee Lake, no enhanced IBRS) is itself an affected part for SPECTRE_V2 and for Intel RETBLEED, so for those two the missing piece is test tooling rather than silicon. What cannot be reached on it:

- **INCEPTION** needs AMD Zen 3 or Zen 4, and the AMD variant of RETBLEED needs Zen 2—both satisfiable by one commodity AM4 desktop.
- **BHI as its own mechanism** needs an enhanced-IBRS-era Intel part; on older silicon it is not separately testable. We note that no automated oracle for it exists to run on such a part either, so the CPU alone would not produce a result.
- **A second ISA with real-silicon ground truth** needs a bare-metal out-of-order Arm Linux host for Revizor's arm64 backend, which supports only the conditional-branch, store-bypass and delayed-exception clauses there.
- **Out-of-order RISC-V** is reachable either in RTL simulation, where published fuzzers already detect several classes on open cores, or on out-of-order RISC-V silicon. Simulation is free and should be tried first.

Full tool-by-tool and part-by-part detail, with citations and prices, is in the repository's oracle-tooling survey and coverage notes.[2]

VII. Related Work

Attacks: Spectre [1] introduced conditional- and indirect-branch speculation attacks; Meltdown [2] and Foreshadow [3] faulting-load transients; RIDL [4] and ZombieLoad [5] microarchitectural data sampling; Retbleed [6] and Inception [7] return speculation; and branch history injection [8] the history-conditioned variant of V2. Our nine classes follow this taxonomy.

Symbolic and static detection: Spectector [10] formalises speculative non-interference and checks it symbolically for conditional-branch speculation; Spectector-Combined [20] extends it to further mechanisms and their combinations. Relational symbolic execution [28], static analysis [29] and speculative taint tracking [30] detect Spectre gadgets in binaries. These tools are our oracles or alternatives to them; none generates gadgets, and each covers a subset of mechanisms.

Fuzzing: SpecFuzz [31] simulates speculation in software to fuzz programs for V1 gadgets. Revizor [12] instead tests the CPU, fuzzing random programs against a speculation contract [9]; we use it as our hardware oracle.

Gadget scanning: The closest work in spirit is Kasper [27], which scans the Linux kernel for "generalized" transient-execution gadgets: it compiles the kernel with a

[2] `docs/ORACLE_TOOLING_SURVEY_2026-10-09.md`, `docs/COVERAGE_AND_NEXT_STEPS_2026-10-09.md`.

DataFlowSanitizer-based toolchain, fuzzes it with a modified syzkaller, and flags a gadget whenever a taint policy connects the three steps of a transient attack—attacker-controlled data on the speculative path, a secret access (out-of-bounds or use-after-free), and a covert channel (cache, MDS, or port contention). Kasper and this work attack the same object from opposite ends. Kasper *discovers* gadgets that already exist in shipped code; we *synthesise* candidate gadgets with a class- and ISA-conditioned generator, so we can produce labelled examples for class/ISA combinations that no single body of real software exposes on demand (rare classes, riscv64), at the cost of the real secret→transmit context a kernel gadget carries. The verdict differs in the same way: Kasper's "is this a gadget" decision *is* its taint model, which can over- or under-approximate, whereas every leak verdict here comes from an oracle external to the generator and the detector (Spectector's symbolic non-interference, InvisiSpec/gem5, Revizor on silicon), trading Kasper's end-to-end realism for an independent ground truth. Kasper's control/secret/channel decomposition is also a useful lens on our nine-class scheme—its covert-channel axis maps onto our L1TF and MDS classes—and a principled reference for what a secret-source/transmit dataflow feature should capture. Kasper is x86-64 and kernel-only, dynamic and fuzz-driven, and emits no per-variant labels, so it is a conceptual contrast, not a component we reuse.

Feedback-guided search: Our generate–filter–oracle–retrain loop and its pre-oracle gate draw their search strategy from taint- and feedback-guided fuzzing, in the lineage of VUzzer [32], where taint feedback steers mutation toward the input bytes that drive new behaviour; here the analogue is detector/oracle feedback selecting which candidates and mutations to pursue rather than sampling blindly.

Learned detection and generation: FastSpec [24] is the closest learned work: it generates Spectre V1 gadgets with a generative model and detects them with neural embeddings. We extend the idea to nine classes and three ISAs, but differ in where the ground truth comes from: FastSpec's generated gadgets are labelled by construction, whereas ours are labelled by external oracles, which is what exposes the gap between "class-shaped" and "leaking" (Section III-D).

Simulation: InvisiSpec [11] is a defence implemented in gem5; we use its 2019 fork, whose classic caches leave a misspeculated-load footprint that stock gem5 does not (Section II-B).

VIII. Conclusion

SpecExec has a working oracle-verified generation loop on x86 that finds many distinct leaking gadgets—at a confirmed-leak yield of about 1.0 for Spectre V1 and 0.98 for Spectre V4—with a learned gate in front of the oracle that spends almost every allowed call on a leak. It also has a detector whose headline accuracy we have taken apart: screening, opcode shortcuts, length, the instruction-set confound and synthetic-to-real transfer each changed what the numbers mean.

TABLE XV
Baseline vs. pretrained generator after oracle-in-the-loop fine-tuning (Spectre V1, x86, 5 seeds, mean ± 95% CI).

Metric	Baseline	Pretrained
Top-1 template mult.	28.8 ± 6.3	**3.0 ± 1.1**
Unique leaking gadgets	102.4 ± 18.0	**134.8 ± 5.9**
Unique rate	0.674 ± 0.083	**0.971 ± 0.021**
Round-0 yield	0.540	0.545
Overall yield	**0.839 ± 0.078**	0.703 ± 0.030

The most solid new result is a property of the data, not of a model: on real hardware, a single `lfence` at the speculation boundary removes the leak in every stable case—on silicon for the three classes whose recorded violations reproduce (L1TF, Spectre V1, Spectre V4)—while fences elsewhere do not. Four things remain open: a mitigation-aware model has to beat the non-learned fence rules on hardware labels, minimality has to be measured on complete programs, Spectre V2 positives have to come from hardware rather than synthesis, and verification has to reach a second ISA. Those steps would turn the pipeline from recognition into discovery.

Appendix A
Generation: development and ablations

The main text (Sec. V-D3, Table VIII) reports the final generator configuration and its confirmed-leak yield. This appendix records the ablations and the per-class/per-ISA development behind those numbers.

A. Self-supervised pretraining: a diversity/yield trade (n=5)

Two bugs initially made pretraining look useless:

- **Tokeniser mismatch**: only 4 of 460 embeddings transferred (322/460 after the fix).
- **Fine-tune learning rate**: at 3×10^{-3} the fine-tune overwrote the pretrained weights (fixed at 10^{-4}).

With both fixed, Table XV shows that pretraining strongly reduces mode collapse and increases distinct leaking gadgets, at a significant cost in raw yield. For discovery, where many distinct gadgets beat repeated copies, this is the right trade.

B. Base multi-class loop, before length conditioning

This is the oracle-in-the-loop rejection-sampling loop with `(class, arch)` conditioning only, before the control-flow fixes and the length-conditioning token of Sec. V-D4; it is the baseline the final Spectre V4 yield of 0.98 (Table VIII) should be read against. It also fixes the oracle-coverage boundary: Spectector adjudicates only V1 and V4, so V2 and Retbleed moved to other oracles (V2 later to Spectector-Combined, Section V-D6; Retbleed is still open, Sec. VI-C). Across 13 runs (about 2,600 generations) the loop produced about 2,048 confirmed leaks and about 1,594 distinct leaking gadgets (counted within each run).

TABLE XVI
ORACLE-IN-THE-LOOP YIELD PER CLASS (SPECTECTOR, x86, 3 SEEDS, MEAN ± 95% CI).

Metric	V1	V4	V2	Retbleed
Adjudicable	0.82	0.87	0.15	0.002
Adjudicated yield	0.998	0.648	0.00	—
Unique leaking gadgets	106 ± 51	76 ± 47	0	0

C. Per-ISA architectural validity: what was broken, and what fixed it

No *leak* oracle covers ARM64 or riscv64, so every leak number in this work is x86. What can be measured on any ISA is whether the generator's output is real, executable machine code—a necessary condition for a leak, and previously unmeasured outside x86: the only ARM64 quality figure was a classifier-judged "looks like class X" hit rate, and the ISA-purity figure of 96–98% is known to be optimistic because the purity metric was blind to ARM load/store mnemonics appearing in x86 output. Measuring it through the gate of Section III-F1 (realised, assembles, executes; deliberately *not* a leak rate) showed that the non-x86 arms were limited by the generator rather than by the missing oracle, and in both cases the cause was mechanical.

ARM64: a mask that existed but was never connected: ARM64 sequences assembled 15% of the time. The failures sorted into three buckets: symbol-name tokens emitted as mnemonics (`main_func`), x86 mnemonics leaking into ARM64 output (`pushq`, `movq`, `leave`), and genuine operand-form errors (`mrs x0, #1`). Most of it was a tool the project already had: a sampling mask that admits a token only if the target ISA's specification recognises its opcode. It was wired into one batch-generation script but *not* into the loop that produces our actual results, nor into the decoder. Connecting it gives 0.75.

What survived the specification rule was precisely what a specification cannot know—what one assembler accepts. `bne` is an **ARM32** spelling that reached the ARM64 vocabulary because the pipeline folds arm32 records into arm64, and aarch64 spells it `b.ne`; `add.4s`, `movi.4s` and `and.16b` are Apple-style NEON syntax that the GNU aarch64 assembler rejects. Adding a second rule—keep a token only if it *realises into text the target assembler accepts*, at one assembler call per token per ISA, cached—reaches **0.96**. The honest cost, reported in Table VII as a unique-sequence rate rather than hidden: the mask leaves 99 admissible ARM64 tokens against 234 for x86, so validity is bought partly by restricting the generator.

riscv64: two blockers, found by round-tripping rather than assumed: The committed checkpoint was conditioned on x86_64 and ARM64 only, so riscv64 could not be sampled at all, although the specification, realiser, tokeniser, mask and emulator nominally covered it. Running the real riscv corpus through realise-then-assemble showed the path was broken end to end, and fixing it exposed a second problem that would have silently wasted the retrain.

The realiser produced *nothing* assemblable (0/80 records). RISC-V encodes immediates in narrow fields—I-type takes $[-2048, 2047]$, `lui` a 20-bit value, shifts 0–63—while the attack-critical constant pool is 4096, 256 and `0xff`, so almost every instruction was out of range. Three further causes followed: `call/tail/jal` take a bare symbol, not a PC-relative expression; a branch's last operand is its *target*, so an immediate there must be the self-relative form; and normalisation maps riscv's `%hi(sym)` relocation syntax to a memory operand because it looks like `X(Y)`, yielding the invalid `lui s3, 0(t2)`. Finally `cbo.inval` requires the Zicbom extension—which matters beyond syntax, since it is riscv's cache-maintenance instruction and therefore the corpus's cache-flush primitive for the timing classes. With all five addressed through new specification keys (so no ISA literal enters the code, and x86/ARM64 are untouched by construction), **120/120** corpus records realise into assemblable riscv64.

The second blocker was in the trainer: it tokenised *every* record with the shared base specification, whose addressing grammar does not match riscv. Measured on the riscv training slice, 0/100 realised sequences assemble under the base grammar against 100/100 under riscv's own. A retrain would have built a malformed riscv vocabulary and every downstream riscv number would have failed for the wrong reason. Per-record tokenisation is therefore now an explicit option rather than a silent default, because it also changes about 8.5% of x86 records and hence the vocabulary. Retraining with the riscv corpus—group-split, with the evaluation holdout structurally excluded—puts riscv64 in the vocabulary for the first time.

D. Length and execution-graph-size diversity

The generator is conditioned only on `(class, arch)`, not on size, so a natural question is whether its output spans a range of attack lengths or collapses onto one shape. We sample a pool per class (x86, temperatures $0.8/1.0/1.2$, n=198 realised sequences each), bucket it by instruction count, and measure within-bucket diversity with the same two notions used elsewhere in this paper (Sec. V-D3): exact-dedup unique-rate, and mean pairwise Ruzicka (multiset-Jaccard) *similarity* over the unique set, where a lower value means more diverse. The instruction count and the program dependence graph's node count coincide here (one node per instruction), so the same buckets index both "length" and "execution-graph size".

Table XVII shows the generator already populates short, medium and long gadgets for both classes, and that the pooled set is more diverse than any single length: mean pairwise similarity for the full range is below every individual bucket's (0.136 vs. ≥ 0.167 for V1; 0.193 vs. ≥ 0.243 for V4), which—unlike a raw count of distinct sequences—does not inflate with sample size. Spanning all buckets also yields more distinct gadgets than the single richest one (163 vs. 94 for V1, 129 vs. 98 for V4), though that comparison partly reflects the larger pooled sample and we do not lean on it. Breadth is *not* free:

TABLE XVII
DIVERSITY BY LENGTH BUCKET (X86). SIMILARITY IS MEAN PAIRWISE RUZICKA OVER THE UNIQUE SET; LOWER IS MORE DIVERSE. THE POOLED ROW'S SIMILARITY FALLS *BELOW* EVERY SINGLE BUCKET'S, SO THE GAIN IS NOT MERELY A LARGER SAMPLE.

Class	Bucket (instrs)	n	unique	uniq-rate	sim↓
V1	S (≤ 8)	11	9	0.818	0.235
	M (9–16)	75	60	0.800	0.240
	L (> 16)	112	94	0.839	0.167
	all	198	163	0.823	**0.136**
V4	S (≤ 8)	10	6	0.600	0.536
	M (9–16)	35	25	0.714	0.278
	L (> 16)	153	98	0.641	0.243
	all	198	129	0.652	**0.193**

oracle yield is length-dependent, and for Spectre V4 the leaks concentrate at longer lengths (Section V-D4). One bucket the model under-fills is short V1 (it has a natural length floor); the length-conditioning token of Section V-D4 both controls length directly and makes that length–yield dependence a usable knob.

APPENDIX B
INFRASTRUCTURE AND REPRODUCIBILITY

The source file in the repository behind every table is recorded in a comment next to that table in the LaTeX source.

Spectector-Combined toolchain: The artifact does not run against current Ciao, and the fix was to pin Ciao to the fork's era. Built FROM the base image's current Ciao, the fork compiles—`semanticsv2.pl` present, `-v 2` accepted—but its binary fails at run time on even a two-line program, with and without `-v` and on the fork's own bundled tests, while the upstream image analyses the same victim correctly. Diagnosing it as version drift (the fork is a 2022 artifact requiring only `core` $\geq$ 1.18) and rebuilding against Ciao v1.22.0-m1 (2022-09-28) resolves it: the minimal program returns SAFE and the fork's V2 example returns a data leak with well-formed statistics. `Dockerfile.spectector_combined_pinned` records the recipe, and its build self-tests the V2 example so a green build is a working oracle.

Our oracle's call path also needed the flags the fork's own scripts use on the extended analyses—an explicit entry point (auto-detected from the compiled `.s`), `--skip-uns --parse-uns`, and a speculative window—gated so the upstream V1/V4 path is byte-identical to before.

Reaching a first Spectre V2 verdict: Reaching this number was almost entirely an infrastructure problem, which we record because it is the kind of silent failure that otherwise corrupts a result: on a cluster whose `/disk/scratch` is node-local the oracle image was invisible to the compute node and every gadget read UNRUNNABLE, and the fork's default million-step budget made the `-v 2` analysis non-halting on any looping victim; a staged image, the 50-instruction window, a 20,000-step and 30-second per-gadget cap, and a combined-image availability check were all required before a single real verdict appeared. Each of those first presented as "0% yield," indistinguishable at the surface from the true low yield until isolated.

APPENDIX C
DEVELOPMENT HISTORY (V54/V55)

These are the in-distribution results reported in earlier drafts. They predate the screening fix (Sec. IV-A2) and the spec bug fixes (Sec. V-A), and should be read as development history.

TABLE XVIII
DATASET STATISTICS, V55 (TRAINING AND LOCKED TEST).

Class	Train	Test	Total
BENIGN	2840	1031	3871
BHI	318	67	385
INCEPTION	518	94	612
L1TF	582	37	619
MDS	445	45	490
RETBLEED	298	75	373
SPECTRE_RSB	470	1	471
SPECTRE_V1	644	42	686
SPECTRE_V2	485	154	639
SPECTRE_V4	347	124	471
Total	6947	1670	8617

v54 vs. v55: Locked-test accuracy was 95.75% (v54) and 95.45% (v55, best of 3; range 93.41–95.45%). Fixing the augmentation raised Spectre V2 recall from 76.0% to 87.7% and Retbleed from 90.7% to 97.3%. L1TF recall fell from 81.1% to 62.2% (37 test records; 62–76% across seeds).

Augmentation bugs fixed by the test suite:

- Register-pool collision in rename (two registers could map to one).
- x86 conditional branches unguarded in local swap.
- Flag-clobber regex missed size suffixes.
- Barrier synonyms allowed downgrades (`dsb sy` → `dsb ish`).
- x86 load regex missed suffixed moves.
- Branch-flip guard missed `jmpq */callq *`.

Learned edge scales: SPEC_RETURN is the most amplified edge type ($s \approx 1.23$–1.25) and CONTROL_FLOW the most suppressed ($s \approx 0.74$). An edge ablation over 5 seeds shows that removing the speculative edge types costs about 0 pp of accuracy but -3.75 pp of macro-F1. The taxonomy matters for minority classes.

APPENDIX D
INLINE FEATURES

The 58 inline features fall into seven groups:

- opcode fractions for 28 security-relevant mnemonics;
- six structural ratios;
- seven normalised pattern counts;
- ten primitive-presence flags;
- four ratios (e.g. `ret_call_ratio`);
- one caller-context flag;

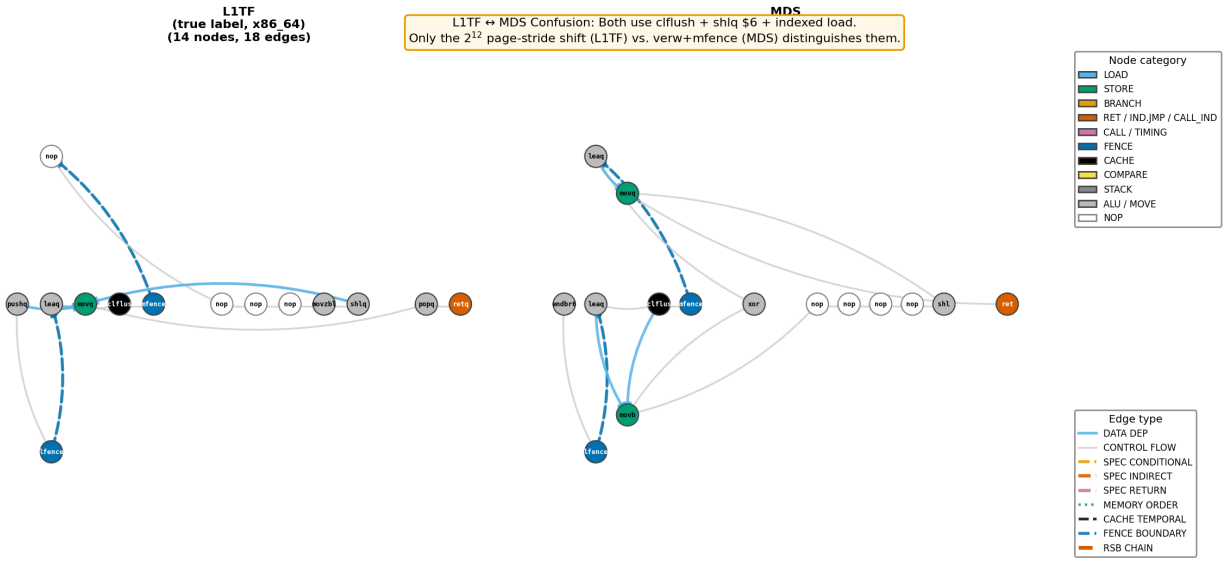


Fig. 2. L1TF vs. MDS PDGs (x86, Clang): both follow `clflush` → `nop`$^{+}$ → shift → indexed load. The discriminating page-stride shift or `verw` is absent in short windows.

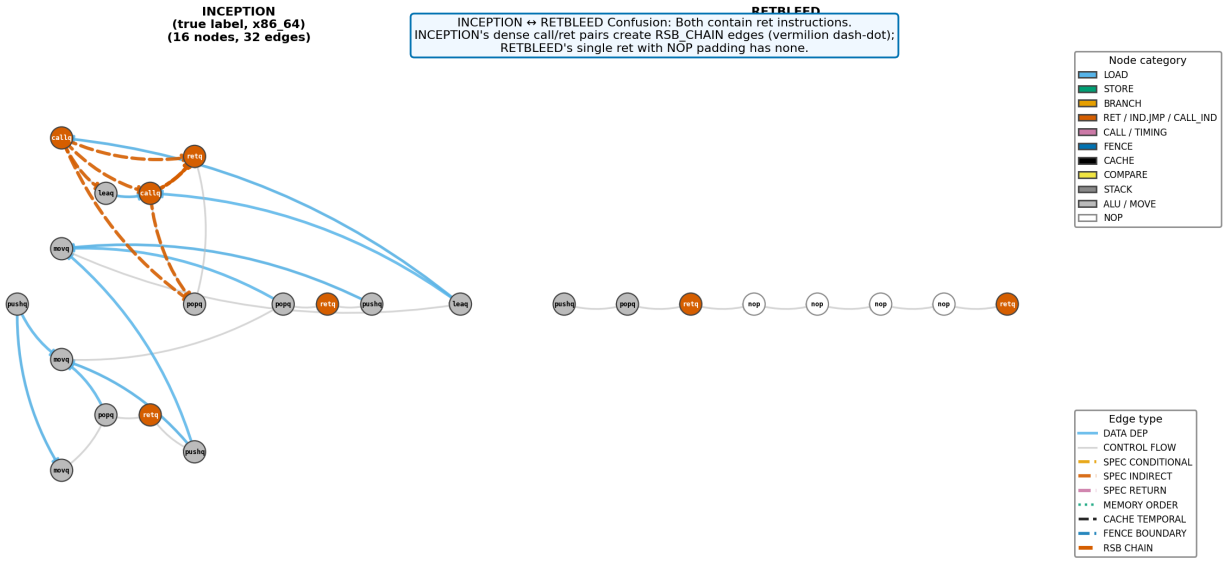


Fig. 3. Inception (dense RSB_CHAIN subgraph) vs. Retbleed (near-linear chain with an isolated `ret`).

- two page-probe features (a `shl $9--15` followed by an indexed load; `$6` is a cache-line probe shared by MDS and L1TF).

`endbr64` was excluded after it increased Retbleed confusion.

References

[1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *IEEE Symposium on Security and Privacy (S&P)*, 2019.

[2] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading kernel memory from user space," in *USENIX Security Symposium*, 2018.

[3] J. Van Bulck *et al.*, "Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution," in *USENIX Security Symposium*, 2018.

[4] S. van Schaik *et al.*, "RIDL: Rogue in-flight data load," in *IEEE Symposium on Security and Privacy (S&P)*, 2019.

[5] M. Schwarz *et al.*, "ZombieLoad: Cross-privilege-boundary data sampling," in *ACM Conference on Computer and Communications Security (CCS)*, 2019.

[6] J. Wikner and K. Razavi, "RETBLEED: Arbitrary speculative code execution with return instructions," in *USENIX Security Symposium*, 2022.

[7] D. Trujillo, J. Wikner, and K. Razavi, "Inception: Exposing new attack surfaces with training in transient execution," in *USENIX Security Symposium*, 2023.

[8] E. Barberis *et al.*, "Branch history injection: On the effectiveness of hardware mitigations against cross-privilege Spectre-v2 attacks," in

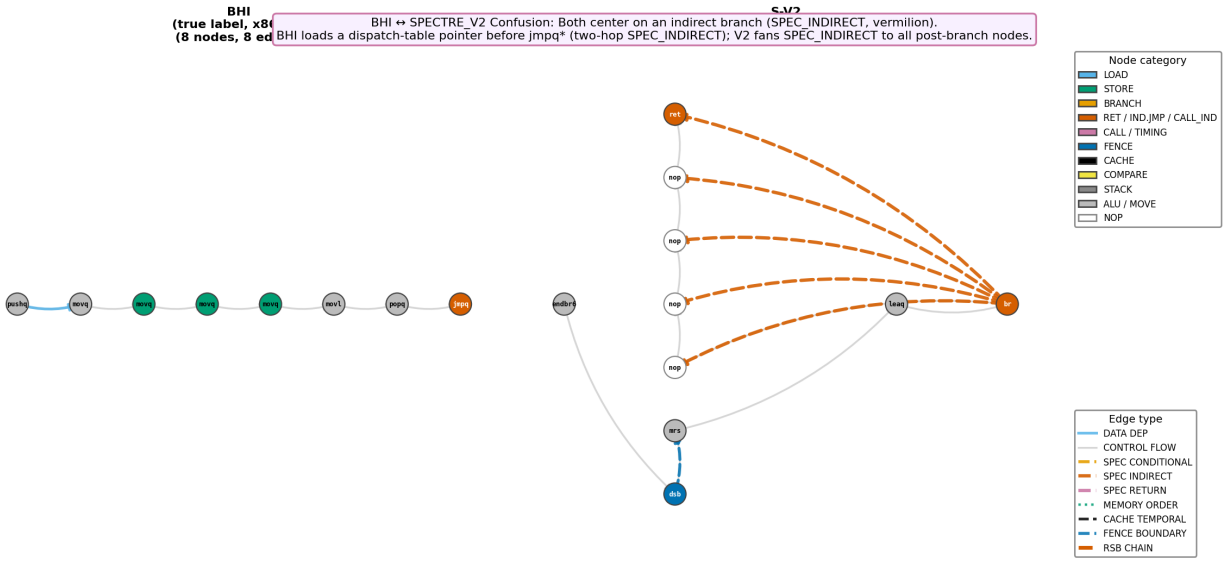


Fig. 4. BHI vs. Spectre V2: both dominated by SPEC_INDIRECT edges; BHI has a preceding dispatch-table load chain.

USENIX Security Symposium, 2022.

[9] M. Guarnieri, B. Köpf, J. Reineke, and P. Vila, "Hardware-software contracts for secure speculation," in *IEEE Symposium on Security and Privacy (S&P)*, 2021.

[10] M. Guarnieri, B. Köpf, J. F. Morales, J. Reineke, and A. Sánchez, "Spectector: Principled detection of speculative information flows," in *IEEE Symposium on Security and Privacy (S&P)*, 2020.

[11] M. Yan, J. Choi, D. Skarlatos, A. Morrison, C. Fletcher, and J. Torrellas, "InvisiSpec: Making speculative execution invisible in the cache hierarchy," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018.

[12] O. Oleksenko, C. Fetzer, B. Köpf, and M. Silberstein, "Revizor: Testing black-box CPUs against speculation contracts," in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.

[14] W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec, "Strategies for pre-training graph neural networks," in *International Conference on Learning Representations (ICLR)*, 2020.

[15] K. Xu, C. Li, Y. Tian, T. Sonobe, K.-i. Kawarabayashi, and S. Jegelka, "Representation learning on graphs with jumping knowledge networks," in *International Conference on Machine Learning (ICML)*, 2018.

[16] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[17] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[19] A. F. da Silva, B. C. Kind, J. W. de Souza Magalhães, J. N. Rocha, B. C. F. Guimarães, and F. M. Q. a. Pereira, "AnghaBench: A suite with one million compilable C benchmarks for code-size reduction," in *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021.

[20] X. Fabian, M. Guarnieri, and M. Patrignani, "Automatic detection of speculative execution combinations," in *ACM Conference on Computer and Communications Security (CCS)*, 2022.

[21] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, 2002.

[22] G. Wang, R. Shen, J. Chen, Y. Xiong, and L. Zhang, "Probabilistic delta debugging," in *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021.

[23] Google, "SafeSide: Understand and mitigate software-observable side-channels." https://github.com/google/safeside, 2019.

[24] M. C. Tol, B. Gulmezoglu, K. Yurtseven, and B. Sunar, "FastSpec: Scalable generation and detection of spectre gadgets using neural embeddings," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2021.

[25] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *USENIX Security Symposium*, 2022.

[26] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, no. 6, 1987.

[27] B. Johannesmeyer, J. Koschel, K. Razavi, H. Bos, and C. Giuffrida, "KASPER: Scanning for generalized transient execution gadgets in the Linux kernel," in *Network and Distributed System Security Symposium (NDSS)*, 2022.

[28] L.-A. Daniel, S. Bardin, and T. Rezk, "Hunting the haunter—efficient relational symbolic execution for Spectre with Haunted RelSE," in *Network and Distributed System Security Symposium (NDSS)*, 2021.

[29] G. Wang *et al.*, "oo7: Low-overhead defense against Spectre attacks via program analysis," *IEEE Transactions on Software Engineering*, 2021.

[30] Z. Qi *et al.*, "SpecTaint: Speculative taint analysis for discovering Spectre gadgets," in *Network and Distributed System Security Symposium (NDSS)*, 2021.

[31] O. Oleksenko *et al.*, "SpecFuzz: Bringing Spectre-type vulnerabilities to the surface," in *USENIX Security Symposium*, 2020.

[32] S. Rawat, V. Jain, A. Kumar, L. Cojocar, C. Giuffrida, and H. Bos, "VUzzer: Application-aware evolutionary fuzzing," in *Network and Distributed System Security Symposium (NDSS)*, 2017.